VectraYX-Nano: A 42M-Parameter Spanish Cybersecurity Language Model with Curriculum Learning and Native Tool Use

Juan S. Santillana
Globant*
juan.salas@globant.com

Abstract

We present VectraYX-Nano, a 41.95M-parameter decoder-only language model trained from scratch in Spanish for cybersecurity, with a Latin-American regional focus and native tool invocation via the Model Context Protocol (MCP). The model is built around four contributions. **(i) Corpus.** VectraYX-Sec-ES, a 170M-token Spanish corpus assembled by an eight-VM distributed pipeline at ~$25 USD of cloud compute and partitioned into three curriculum phases: conversational (42M tokens, OpenSubtitles-ES [35] and OASST1 [33]), cybersecurity (118M tokens, NVD [39], Wikipedia-ES, in-house NVD-derived Spanish CVE mirror, security blogs), and offensive-security tooling (10M tokens, ExploitDB, HackTricks, OWASP). **(ii) Architecture.** A 42M-parameter Transformer decoder combining Grouped-Query Attention [2], QK-Norm [14], RMSNorm [58], SwiGLU [49], RoPE [52], and a z-loss auxiliary [11], paired with a domain-balanced 16,384-token byte-fallback BPE [34, 48] trained on a 50/50 conversational/technical mixture. **(iii) Curriculum with replay.** Continual pre-training across the three phases with a replay buffer [30] mitigates catastrophic forgetting [19, 32] and yields a monotonic loss descent ($9.80 \rightarrow 3.17 \rightarrow 3.00 \rightarrow 2.16$). After SFT (final loss 1.74) on a curriculum-aware mixture of OASST-ES, Alpaca-ES, CVE Q&A, and tool-use traces, the v2 bootstrap-ablation reference attains a conversational gate of 0.775 ± 0.043 on B5 over $N = 4$ seeds (Section 8.6), and a controlled Phase-2 replay sweep over $\{0, 5, 10, 25, 50\}\%$ saturates B5 at $\geq$ 25% replay (Section 6.8). **(iv) Two empirical findings, both N=4.** A controlled bootstrap-corpus ablation across v2 (OpenSubs), v4 (mC4-ES [57]), and v6 (60/25/15 OpenSubs/mC4/Wiki) under $N = 4$ seeds exposes a *loss-versus-register inversion*: the lower-perplexity bootstraps yield measurably worse conversational behavior ($v2 > v4 > v6$ on B5 at every paired seed), indicating that at the nano scale the bootstrap-corpus register dominates downstream chat quality. The B4 (tool-selection) floor of 0.000 on the mixed SFT corpus is a *corpus-density artifact*, not a capacity gate: rebalancing the SFT mixture to tool-use ratio 1:21 yields VectraYX-Nano v7, the released headline configuration, which reaches $B4 = 0.230 \pm 0.052$ at 42M parameters ($N = 4$ seeds) while retaining $B1 = 0.332 \pm 0.005$ and $B5 = 0.725 \pm 0.130$ (Section 8.7); a LoRA [29] replication on a 260M from-scratch mid-tier reaches 0.445 ± 0.201. The released GGUF [22] artifact is 81 MB in F16 (approximately 20 MB in 4-bit quantization), runs at sub-second time-to-first-token on commodity hardware under `llama.cpp` [21], and is, to the best of our knowledge, the first published Spanish-native cybersecurity LLM with end-to-end MCP integration. We release the corpus construction recipe, training scripts, configurations, GGUF weights, and the B1–B5 benchmark suite (B1: 500, B2: 200, B3: 100, B4: 200, B5: 314 prompts) for reproducibility.

CCS Concepts

• **Computing methodologies** → **Natural language generation**; *Neural networks*; • **Security and privacy** → **Software and application security**.

Keywords

Language models, Cybersecurity, Spanish NLP, Curriculum learning, Tool use, Model Context Protocol, Edge inference

1 Introduction

Large language models (LLMs) have become a foundational tool for security analysts: they assist in vulnerability triage, log analysis, malware classification, and incident response. However, the publicly available LLM ecosystem suffers from two well-documented coverage gaps that compound when intersected. First, the strongest open-weight chat models are trained predominantly on English text [5, 44, 53], with Spanish typically representing a small fraction of pre-training mixtures, despite Spanish being the second-most-spoken native language in the world [18]. Second, while there is a growing literature on cybersecurity-specialized language models, virtually all of these models are trained on English corpora [1, 6] and none, to our knowledge, target Latin-American security terminology, regional CSIRT vocabularies (CCN-CERT, INCIBE, CSIRT-CL, COLCERT), or the LATAM threat-intelligence context.

These two gaps are jointly painful for security operations centers (SOCs) in Latin America. Spanish-speaking analysts who would otherwise benefit most from LLM assistance must work either with English-only domain models, with general-purpose Spanish models that lack technical accuracy, or with frontier closed-source models whose behavior they cannot audit, retrain, or deploy on-premise. The on-premise constraint is not academic: LATAM security teams routinely process classified incident reports, customer PII, and unreleased indicators of compromise that cannot leave the network.

A second motivation for this work is the rise of *tool-augmented* language models [41, 43, 47] and, more recently, the emergence of the Model Context Protocol (MCP) [4] as a standard for LLM–tool interfacing. Cybersecurity is one of the strongest application domains for tool use, since the underlying knowledge changes daily (new CVEs, KEV additions, TTP updates) and an analyst's typical query ("is this CVE being exploited?", "has this hash been flagged?") has an authoritative external answer that a parametric model cannot reliably memorize. A small parametric model that knows *when* to call a tool can be substantially more useful than a much larger model that hallucinates answers from a frozen training cutoff.

*The author is employed at Globant. Institutional affiliation approval is pending.

Contributions. We present VectraYX-Nano, a 41.95M-parameter Spanish-language cybersecurity LLM trained from scratch with native MCP tool-use support. Our contributions are:

(1) **VectraYX-Sec-ES corpus.** We release the construction recipe for a 170M-token Spanish cybersecurity corpus assembled from a distributed pipeline of eight virtual machines. The corpus includes 88K NVD CVE entries, 50K previously translated Spanish CVEs from an in-house NVD-mirror SQLite store, a 53,590-article filtered Spanish Wikipedia subset (82M tokens, the single largest component), translated ExploitDB entries, the Spanish translations of HackTricks and OWASP, and a curated set of conversational Spanish from OpenSubtitles-ES [35] and OASST1 [33]. The full pipeline costs approximately $25 USD in cloud compute.
(2) **Modern small-LLM architecture.** We design a 41.95M-parameter Transformer decoder that integrates Grouped-Query Attention [2], QK-Norm [14], RMSNorm [58], SwiGLU [49], RoPE [52], weight-tied embeddings, and a z-loss auxiliary [11], alongside a domain-balanced 16,384-token byte-fallback BPE [34, 48] tokenizer trained on a 50/50 conversational/technical mixture.
(3) **Curriculum pre-training with replay.** We apply a three-phase curriculum (conversational → cybersecurity → tooling) with explicit replay buffers between phases following [30]. The phase weights are 100% conversational → 75%/25% tech/conv → 70%/20%/10% tools/tech/conv. The pre-training loss decreases monotonically across phases (9.80 → 3.17 → 3.00 → 2.16) without observable catastrophic forgetting [19, 32].
(4) **Tool-use supervision via MCP.** We construct a 6,327-example tool-use SFT dataset templated against a real on-premise CVE database (50K Spanish CVEs, 27K exploits, 98K IOCs) and bound to six MCP servers (NVD, CISA KEV, MITRE ATT&CK, OTX, LATAM intel, bash execution). The model learns to emit grammatically correct <|tool_call|> JSON segments that the MCP runtime executes verbatim.
(5) **Curriculum ablation: bootstrap-corpus register matters ($N = 4$).** We report a controlled ablation that swaps OpenSubtitles-ES (Phase 1, v2) for mC4-ES [57] filtered with FineWeb-2 [42] quality scores (Phase 1, v4), and for a 60/25/15 mixture of OpenSubs / mC4-ES / Wikipedia-ES (v6). All three configurations are retrained from scratch under four independent seeds ($\{42, 7, 13, 23\}$). The mC4-ES variant achieves consistently *lower* loss in every subsequent phase (−0.29 in Phase 2, −0.28 in Phase 3, −0.17 in SFT) but consistently *worse* conversational behavior on the B5 held-out chat gate: 0.775 ± 0.043 (v2) vs. 0.700 ± 0.122 (v4) vs. 0.675 ± 0.083 (v6). The $v2 > v4 > v6$ ordering is preserved at every paired seed comparison. We attribute this inversion to a register-mismatch effect: at the 42M-parameter scale, the bootstrap corpus dictates the model's default response style, and an encyclopedic web register cannot be reliably overwritten by SFT alone.
(6) **Tool-use density threshold; VectraYX-Nano v7 as the released headline model.** The B4 (tool-selection) floor of 0.000 across all three bootstrap configurations of the mixed-SFT v2 reference is a corpus-density artifact, not a capacity gate. Rebalancing the SFT mixture to a tool-use ratio of 1:21 yields VectraYX-Nano v7, the released headline configuration, which reaches B4 = 0.230±0.052 at 42M parameters under $N = 4$ seeds while retaining B1 = 0.332 ± 0.005 and B5 = 0.725 ± 0.130. A separate LoRA [29] study reproduces the density threshold at 1:211 → 1:21: at 1:21 a rank-16 LoRA reaches 0.145 ± 0.046 on Nano and 0.445 ± 0.201 on a 260M from-scratch mid-tier we call VectraYX-Base (N=4 seeds each). The capacity gate is therefore a first-token prior conflict, not a parametric limit.
(7) **Edge-deployable artifact.** We export the fine-tuned model to GGUF [22] (F16: 81 MB; Q4: ∼20 MB), runnable under Ollama [40] or `llama.cpp` [21] with sub-second time-to-first-token on a Raspberry Pi 4. The artifact includes weight-tied LM heads and the 25 reserved domain tokens.

Scope. VectraYX-Nano is positioned as a *nano-scale* model: it is meant to assist analysts on edge devices and in air-gapped environments, not to compete with frontier 70B+ chat models on open-domain reasoning. Within its target envelope — Spanish cybersecurity Q&A, CVE summarization, threat classification, command completion, and tool dispatch — we show that a careful corpus, a domain-balanced tokenizer, and curriculum pre-training with replay can extract qualitative behavior that a similarly sized monolithic pre-training run cannot.

Reproducibility. All training scripts, configuration files, the curriculum sampler with replay-buffer code, the benchmark harness, the tool-use corpus, and the B1–B5 evaluation datasets are released at https://github.com/vectrayx/vectrayx-nano-paper. Model checkpoints and LoRA adapters are available at https://huggingface.co/jsantillana/vectrayx-nano. Section 6 provides exact hyperparameters and Section 8 documents the held-out evaluation protocol. The corpus itself is partially released under upstream licenses (NVD, Wikipedia, ExploitDB, OpenSubtitles); the LATAM-curated portion is released as construction recipes rather than raw text, in line with current practice for security corpora.

2 Related Work

Security-domain language models. Domain-specialized models for security have a short but active history. SecureBERT [1] continues pre-training a RoBERTa [37] backbone on cybersecurity text and reports gains on entity recognition over generic BERT [17]. CySecBERT [6] similarly continues from BERT on a 670K-document English security corpus and improves classification benchmarks. Earlier work in this line includes SciBERT [7], which establishes the methodology of vocabulary-extending continual pre-training for technical domains. All of these models share two limitations from our perspective: they are encoder-only, and they are trained on English. We are not aware of any prior decoder-only generative model published with a Spanish cybersecurity specialization.

Spanish-language and multilingual models. The Spanish NLP ecosystem has matured around BETO [10] (a Spanish BERT), the RoBERTa-base-BNE family [25], and more recently Salamandra [26] from the Barcelona Supercomputing Center, an open Iberian decoder family. mC4 [57] and CC-100 [12] have served as the standard

Spanish pre-training corpora; FineWeb-2 [42] is a more recent multilingual quality-filtered web release. These resources have unlocked general-purpose Spanish language modeling, but none of them targets a security domain or ships with a tool-use modality.

Small / nano-scale language models. Several recent works show that careful data curation can produce competitive sub-billion-parameter models. SmolLM2-135M and SmolLM2-360M [3] achieve strong nano-scale benchmarks via a recipe of high-quality web, code, and synthetic data. MobileLLM [36] establishes that depth is more useful than width at the sub-billion scale and that grouped attention combined with weight sharing is effective. TinyLlama [59] reports that small models can be trained substantially past the Chinchilla [28] optimum and continue to improve. We adopt several of these design intuitions (depth ≥ width, GQA, weight tying) and constrain ourselves to a single GPU training budget.

Tool-augmented and tool-using LLMs. Toolformer [47] introduced self-supervised insertion of tool calls; Gorilla [41] demonstrated retrieval-anchored API calling at training time; ToolLLM [43] curated 16K APIs with rich tool-use traces. More recently, Anthropic's Model Context Protocol (MCP) [4] has emerged as a standard for tool definitions and stateful tool sessions in production deployments. Tool-augmented work in security exists [15], but those systems orchestrate frontier models behind agentic loops; they do not train a small native tool-using model. To our knowledge, VectraYX-Nano is the first nano-scale Spanish security model with native tool-call generation evaluated against a held-out tool-selection benchmark.

Continual pre-training and replay. Continual pre-training without replay tends to suffer from catastrophic forgetting of pre-training distribution, an issue documented since at least [19] and given a Bayesian formulation in elastic weight consolidation [32]. Recent work on continual LLM pre-training [24, 30] shows that simple strategies — learning-rate re-warmup, modest replay percentages from the previous mixture, and adaptive token-budget allocation — recover most of the lost performance with minimal overhead. We follow [30] closely and apply replay percentages of 25% (Phase 2) and 10% (Phase 3), which we find to control the loss trajectory but, as Section 6 reports, materially affect the model's final stylistic register.

Curriculum learning. Curriculum learning [8] predates the LLM era; it has been shown to help under specific data-quality regimes [50]. For LLM pre-training, ordering by difficulty or domain has had mixed results [56], with the more reliable finding being that data *mixture* matters more than data *ordering*. Our setting is different: we are interested in instilling a stylistic register (chat-like Spanish) before specializing in a technical domain, and we report (Section 6) that for nano-scale models the ordering effect is large enough to flip user-visible behavior even when held-out perplexity moves the other way.

Edge-deployable language models. On the deployment side, GGUF [22] and `llama.cpp` [21] have made CPU inference of quantized small LLMs practical on commodity hardware including Raspberry Pi-class devices. Ollama [40] provides a developer-facing interface on top. We export to GGUF and ship a Modelfile so that VectraYX-Nano runs out of the box in this stack.

Positioning. The intersection of (i) Spanish, (ii) cybersecurity, (iii) decoder-only / chat, (iv) trained from scratch, and (v) native MCP tool use — to our knowledge — is empty in the prior literature. The closest single-axis neighbors are SecureBERT [1] (security, English, encoder, no tools), Salamandra [26] (Spanish, general-purpose, decoder, no tools), and Gorilla [41] (English, general-purpose, decoder, tool use). VectraYX-Nano populates the intersection.

3 The VectraYX-Sec-ES Corpus

3.1 Design goals

The corpus is designed to satisfy three constraints simultaneously: (i) sufficient Spanish-language conversational coverage to bootstrap chat behavior in a from-scratch model, (ii) substantive cybersecurity domain coverage in Spanish, including LATAM-specific vocabulary, and (iii) tool-use supervision grounded in a real CVE/IOC database rather than synthetic API descriptions. The corpus totals approximately 170M tokens after deduplication and is partitioned into three shards aligned with the curriculum phases: `phase1_conv`, `phase2_tech`, and `phase3_tools`.

3.2 Distributed collection pipeline

The corpus was assembled by an eight-VM pipeline running in parallel for two days across two clouds (GCP and Azure), at an aggregate compute cost of approximately $25 USD. The eight workers and their roles are:

- `corpus-nvd`: NIST National Vulnerability Database REST API [39]; output 87,998 CVE records (~11.4M tokens).
- `corpus-wiki`: MediaWiki API for the Spanish Wikipedia security categories.
- `corpus-web`: 15 RSS feeds from Spanish-language security blogs.
- `corpus-tech`: 7 RSS feeds from Spanish technology blogs.
- `corpus-papers`: English security paper RSS feeds with Ollama-based translation [40].
- `corpus-malware`: Malpedia and MITRE ATT&CK [51] surfaces with translation.
- `corpus-exploitdb`: ExploitDB GitLab mirror with translation, yielding 2,385 translated entries.
- `corpus-tools`: Eighteen GitHub repositories of pentesting tool documentation (Spanish branches).

On the central training node, additional sources are integrated locally: a 313 MB Wikipedia-ES dump processed via a streaming `bz2` + `iterparse` pipeline [55] (53,590 articles passed a 65-keyword cybersecurity filter, total ~82M tokens, the single largest component of the corpus), GitHub clones of HackTricks-ES (952 documents, the `es` branch), HackTricks-Cloud (566), OWASP Top 10 (60), OWASP ASVS (42), OWASP WSTG (151), and PayloadsAllTheThings (139), and a previously assembled SQLite store hosted on an on-premise server we refer to internally as "Pikachu". Pikachu is not a third-party dataset: it is a long-running, locally maintained mirror of the NVD CVE database augmented with exploit, malware, and IOC tables, and an in-house Spanish translation pipeline that pre-translates each NVD entry as it is ingested. At the time of corpus assembly the Pikachu store contained 50,601 Spanish-translated

Table 1: VectraYX-Sec-ES corpus by curriculum phase.

Phase	Tokens	Sources
`phase1_conv`	42.4M	OpenSubtitles-ES [35], OASST1-ES [33], custom
`phase2_tech`	117.7M	NVD [39], Pikachu (NVD-mirror), Wikipedia-ES, blogs
`phase3_tools`	10.1M	HackTricks-ES, OWASP-ES, ExploitDB, malware
Total	**170.2M**	

CVEs (each derived from its NVD source record), 27,121 exploit entries, 7,556 malware signatures, and 98K IOCs.

3.3 Translation policy

For sources available only in English (papers, ExploitDB entries, malware reports), we translate locally using Ollama's `qwen2.5:1.5b` [44] with temperature 0.1 and a 300-second timeout. We chose the 1.5B variant over the 3B variant after empirical comparison: 1.5B was approximately twice as fast and exhibited a lower timeout rate on long technical documents, with no observable degradation on chunked translations under 2,000 characters. Above this threshold we observed approximately 5–10% mistranslation on highly technical text, and Section 10 discusses the consequences.

3.4 Phase composition

Table 1 reports the per-phase composition. Tokens are reported under the final 16,384-vocabulary BPE tokenizer.

3.5 Domain breakdown

Table 2 provides a finer-grained breakdown by data source. A key observation is the dominance of the filtered Wikipedia-ES dump (82M tokens, ∼48% of the corpus), which provides a broad foundation in Spanish technical writing across security, cryptography, malware, networking, and operating systems. The NVD CVE descriptions are bilingual (mostly English with some Spanish entries); the Pikachu store contributes 50K *already translated* Spanish CVEs — derived from the same NVD records but pre-translated by Pikachu's in-house ingestion pipeline — that complement the raw NVD text.

3.6 Conversational subcorpus

The conversational subcorpus is intentionally small (42M tokens, ∼25% of the total). We assemble it from three sources: OpenSubtitles-ES from OPUS [35] (16,317 chunks, ∼39M tokens of subtitle dialogue), the Spanish-filtered subset of OASST1 [33] (13,000 conversations, ∼3.5M tokens), and 214 short hand-curated dialogues that establish cybersecurity-specific greetings and disclaimers. Section 6 reports an ablation that swaps the OpenSubtitles component for filtered mC4-ES [57] and finds that, despite mC4-ES yielding lower overall pre-training loss, the OpenSubtitles bootstrap produces measurably better chat behavior.

Table 2: Source-level breakdown of the pre-training corpus.

Source	Records	Tokens	Lang.
Wikipedia-ES (filtered)	53,590	82.0M	ES
NVD CVEs	87,998	11.4M	ES/EN
Pikachu CVEs (NVD mirror, ES)	50,601	8.8M	ES
Pikachu exploits	27,121	3.3M	EN/ES
GitHub repos (markdown ES)	1,884	3.7M	ES
Wikipedia-ES (API, security)	947	2.2M	ES
ExploitDB (translated)	2,385	1.3M	ES
Malware DBs (Malpedia, MITRE)	300	0.7M	ES
Pikachu malware signatures	7,556	0.6M	EN
Papers EN→ES	236	0.6M	ES
Tech blogs ES	322	0.5M	ES
Security blogs ES	241	0.4M	ES
Tools docs ES	65+	0.3M	ES
OpenSubtitles-ES	16,317 chunks	39.0M	ES
OASST1-ES	13,000	3.5M	ES

3.7 SFT corpus

The SFT corpus is separate from pre-training. It contains 13K OASST1-ES conversations, 4,030 CVE Q&A pairs derived from the Pikachu store, 6,327 tool-use traces (Section 7), and 214 custom cybersecurity greetings, for approximately 28M tokens. In the family-scale extension (Section 8), the Qwen2.5 fine-tuning corpus also includes `bertin-project/alpaca-spanish` [9] (51,942 examples) and the Spanish-filtered subset of OpenAssistant OASST2 [33] (18,022 conversations).

3.8 Tokenization and binarization

After training the BPE tokenizer (Section 4), we tokenize each phase into `uint16` memory-mapped binary shards in the style of nanoGPT [31], with each phase occupying its own shard directory so that the curriculum sampler can read them at arbitrary mixture weights without re-tokenizing. The total compressed shard footprint is ∼340 MB. Phase 1 fits in a single shard; Phase 2 spans three shards; Phase 3 occupies one shard.

3.9 Deduplication and quality filtering

We deduplicate at the document level using exact-match hashing on a normalized form (lowercased, whitespace-collapsed). Cross-source deduplication is non-trivial when the same CVE description appears verbatim in NVD, in the Pikachu store, and in the Wikipedia-ES filtered dump; we tolerate this redundancy because the three surfaces correspond to different stylistic registers (terse advisory, translated narrative, encyclopedic prose) and we observe empirically that repeated exposure to the same vulnerability across these registers helps the model produce a more consistent response style at SFT time.

3.10 Licensing and release

The released portions of the corpus respect upstream licenses (CC0/CC-BY for Wikipedia, CC-BY-SA for HackTricks, public domain for NVD, CC-BY-NC for OpenSubtitles, Apache-2.0 for OASST1). The LATAM-curated portion that combines internal

SQLite-derived translations with hand-curated dialogues is released as a construction recipe rather than raw text, in line with current security-corpus practice.

4 Domain-Balanced Tokenizer

4.1 Design rationale

A first iteration of the tokenizer was trained on a 95%-technical mixture and produced a vocabulary that fragmented common Spanish chat tokens (*"hola"*, *"gracias"*, *"estás"*) while merging long technical n-grams. We replaced it with a 50/50 mixture for two reasons. First, the post-tokenization *token budget* for chat sentences directly affects how many gradient updates a chat example contributes to the loss; oversplit chat tokens are a hidden source of register imbalance in the loss. Second, byte-fallback is necessary to guarantee that the model can serialize CVE identifiers, hashes, and base64 payloads without producing <unk> tokens.

4.2 Configuration

We use SentencePiece [34] BPE [48] with the configuration listed below, taken verbatim from `configs/nano.json`:

```
{
  "vocab_size": 16384,
  "model_type": "bpe",
  "character_coverage": 1.0,
  "byte_fallback": true,
  "normalization": "nmt_nfkc",
  "split_digits": true,
  "split_by_unicode_script": true,
  "add_dummy_prefix": true,
  "balance": {
    "conversational_ratio": 0.5,
    "technical_ratio":      0.5
  }
}
```

The training corpus for the tokenizer is a 2M-line balanced sample drawn evenly from `phase1_conv` (chunks from OpenSubtitles-ES and curated dialogues) and `phase2_tech` + `phase3_tools` (chunks from NVD, Wikipedia-ES, HackTricks-ES, ExploitDB, etc.). We use `nmt_nfkc` normalization to preserve Spanish accents (NFKC without NFD decomposition); `split_digits=true` ensures CVE numerics tokenize predictably; `byte_fallback=true` guarantees full UTF-8 coverage.

4.3 Special tokens

The vocabulary reserves 27 user-defined symbols at the start, organized in five groups:

- **Control**: <|pad|> <|bos|> <|eos|> <|unk|> <|sep|> (assigned to slots 0–4).
- **Chat roles**: <|system|> <|user|> <|assistant|> <|end|>.
- **Tool use**: <|tool_call|> <|/tool_call|> <|tool_result|> <|/tool_result|>.
- **Domain**: <|cve|> <|cvss|> <|ioc|> <|ttp|> <|mitre|> <|kev|> <|exploit|> <|patch|> <|alert|>.
- **Severity**: <|critical|> <|high|> <|medium|> <|low|> <|info|>.

Reserving these as user-defined symbols (rather than relying on the BPE merges to discover them) ensures that they always tokenize as exactly one token. The remaining 16,357 vocabulary slots are filled by BPE merges.

4.4 Empirical token economy

On Spanish chat sentences the v2 tokenizer is approximately twice as efficient as the prior v1 technical-only tokenizer. Representative examples (single-token decoding shown):

- `"vulnerabilidad"` → 1 token (v1: 4–5 tokens).
- `"CVE-2021-44228"` → 5 tokens (`CVE`, `-`, `2021`, `-`, `44228`); digit splitting keeps year and ID symbolic.
- `"¡Hola! ¿cómo estás?"` → 9 tokens (v1: ∼18 tokens).
- `"<|user|>¿qué es ransomware?<|end|>"` → 8 tokens (chat role markers as single tokens).

The asymmetric improvement on chat tokens (v1 nearly doubled) is the empirical effect that the design targets.

4.5 Vocabulary size choice

We chose 16,384 (rather than the more common 32K or 50K) on three grounds. (i) At 42M parameters with weight-tied embeddings, the embedding matrix is $16384 \times 512 = 8.4$M parameters, ∼20% of the total budget. Doubling the vocabulary to 32K would push the embedding share to ∼33%, leaving substantially less budget for transformer blocks. (ii) On the deployment side, 16K vocabularies quantize cleanly under GGUF Q4 schemes. (iii) Empirically, the 16K vocabulary covers the corpus with <0.05% byte-fallback rate (i.e., almost all characters tokenize through merges rather than falling back to byte units), which we treat as a sufficient compression target for a domain model.

5 Architecture

5.1 Overall design

VectraYX-Nano is a Transformer [54] decoder-only language model with eight pre-norm blocks. The architecture follows the modern Llama-family stack [5, 53]: RMSNorm [58] for normalization, SwiGLU [49] for the FFN nonlinearity, RoPE [52] for positional encoding, weight-tied input/output embeddings, no biases on linear layers, and a z-loss auxiliary [11] on the logits. We add two stability improvements that have become standard in recent small-LLM releases: Grouped-Query Attention (GQA) [2] with $n_q = 8$ and $n_{kv} = 2$, and QK-Norm [14, 27] that applies RMSNorm independently to query and key projections.

5.2 Hyperparameters

Table 3 reports the architecture configuration as drawn from `configs/nano.json`.

5.3 Grouped-Query Attention

With $n_q = 8$ and $n_{kv} = 2$, each KV head is shared by four query heads. The attention layer's projection footprint is therefore:

- $W_Q \in \mathbb{R}^{512\times512}$ (8 query heads of dim 64),
- $W_K, W_V \in \mathbb{R}^{512\times128}$ (2 KV heads of dim 64),
- $W_O \in \mathbb{R}^{512\times512}$.

Compared to standard MHA at the same width ($W_K, W_V \in \mathbb{R}^{512\times512}$), GQA saves ∼50% of the KV parameters and shrinks the KV cache by 4× at inference time. We compute attention through PyTorch's

Table 3: VectraYX-Nano architecture (`configs/nano.json`).

Hyperparameter	Value
Vocabulary size	16,384
Layers (L)	8
Hidden dimension (d_{model})	512
Query heads (n_q)	8
KV heads (n_{kv})	2
Head dimension (d_h)	64
FFN dimension (d_{ffn})	2,048
Maximum sequence length	1,024
RoPE base (θ)	10,000
RMSNorm ϵ	10^{-6}
Init std (σ)	0.02
Residual init scale	$\sigma/\sqrt{2L} = 0.005$
Dropout	0.0
Tied embeddings	yes
QK-Norm	yes
z-loss coefficient	10^{-4}
Total parameters	**41.95M**

`F.scaled_dot_product_attention` with `is_causal=True`, which dispatches to FlashAttention-2 [13] on supported hardware (NVIDIA L4 in our case). Implementation details are visible in `model/transformer.py`: queries are projected to (B, n_q, T, d_h), keys and values to (B, n_{kv}, T, d_h), and KV is repeated $n_q/n_{kv} = 4$ times along the head dimension before the kernel call.

5.4 QK-Norm

We apply RMSNorm to the query and key projections after RoPE rotation but before the attention dot product. This stabilizes training at small batch sizes and small head counts: empirically the gradient norm trace is smoother and we observed no loss spikes during the 3,519 pre-training steps despite running BF16 without a loss scaler. QK-Norm has been adopted by OLMo [23] and Gemma [20] for similar reasons.

5.5 Initialization

We initialize linear and embedding weights from $\mathcal{N}(0, \sigma^2)$ with $\sigma = 0.02$, and rescale residual-output projections (the `wo` of attention and the `w_down` of SwiGLU) by $\sigma/\sqrt{2L}$ following the GPT-2 scaled init [45]. This keeps the variance of activations bounded as the depth grows and is critical for stable BF16 training with z-loss.

5.6 z-loss

The PaLM z-loss auxiliary [11] regularizes the partition function of the softmax, $z = \log \sum_i \exp(\ell_i)$, by adding $\lambda \cdot \mathbb{E}[z^2]$ to the cross-entropy loss with $\lambda = 10^{-4}$. This term keeps the unnormalized logit magnitudes from drifting upward over long training runs and is a cheap insurance against the well-known instability of bf16 softmax over large vocabularies.

5.7 Total parameter accounting

The 41.95M parameter total decomposes approximately as: embedding/LM-head 8.4M (tied; counted once), 8 transformer blocks of ∼4.2M each (∼33.5M) where each block contains ∼0.6M attention parameters and ∼3.1M FFN parameters, plus ∼30K parameters across the nine RMSNorm gain vectors. We confirmed the total at training startup; the script logs `[model] 41.95M params` at every run.

5.8 Inference profile

At inference, the GQA design and small context (1,024 tokens) keep the KV cache footprint to $L \cdot 2 \cdot n_{kv} \cdot d_h \cdot T \cdot 2$ bytes (BF16) = $8 \cdot 2 \cdot 2 \cdot 64 \cdot 1024 \cdot 2 = 4$ MiB per sequence. After GGUF Q4 quantization, weights occupy ∼20 MB. The combination yields a total resident set of ∼60–80 MB at inference, which fits comfortably in the L1+L2+L3 cache hierarchy of modern x86 CPUs and in the 1 GB RAM budget of a Raspberry Pi 4. We measure 6–10 tokens/s on a Raspberry Pi 4 (Cortex-A72, 4 cores) and 60–100 tokens/s on a contemporary laptop CPU.

5.9 Why this configuration?

Two configuration choices warrant explicit justification. (i) *Depth ≥ width.* Following MobileLLM [36], we prefer 8 layers of width 512 to 4 layers of width 1,024 at the same parameter budget, because depth empirically helps reasoning behavior more than width at sub-100M scales. (ii) *Aggressive GQA ratio.* We use $n_q/n_{kv} = 4$, which is more aggressive than Llama-2-7B's 1:1 but matches Mistral-7B and Llama-3.2 conventions. At our parameter scale the savings from GQA are absolute (4M parameters reclaimed for FFN width), and we did not observe any quality degradation on chat or CVE-Q&A relative to a small ablation pilot run with $n_q/n_{kv} = 2$.

6 Curriculum Pre-training with Replay

6.1 Motivation: failures of monolithic pre-training (v1)

Our first pre-training attempt (v1) followed the conventional small-LLM recipe: a single-phase pre-training over the full technical corpus (142M tokens, two epochs) followed by a multi-stage SFT. The resulting model reached pre-training loss 3.35 and SFT loss 0.315, yet exhibited a striking failure mode: it answered the prompt `"hola"` (Spanish for "hello") with a CVE analysis instead of a greeting. Three increasingly conversational SFT runs (v1: 86K examples, v2: 11K examples, v3: 24K examples mixing OASST1) failed to repair this behavior. We conclude that a model that has not seen sufficient conversational Spanish during pre-training cannot be coerced into chat behavior by SFT alone at the 42M scale — the chat register has to be the model's first language.

6.2 Three-phase curriculum

We address this with a three-phase curriculum:

Phase 1 – conversational bootstrap. 100% `phase1_conv` (42.4M tokens, 2 epochs). The model establishes a default Spanish chat register before encountering any technical text.
Phase 2 – domain immersion. 75% `phase2_tech` + 25% `phase1_conv` replay (117.7M tokens). Continued pre-training resumed from the Phase 1 checkpoint at a lower learning rate. The 25% replay buffer follows [30] and is intended to prevent forgetting of conversational Spanish while the model absorbs CVE/Wikipedia/blog text.

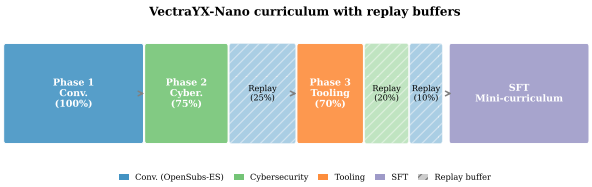

Figure 1: Three-phase curriculum with replay. Each phase samples from a weighted mixture of shard directories; the replay fractions (25% in Phase 2, 10%+20% in Phase 3) prevent catastrophic forgetting of earlier registers.

Phase 3 – tooling specialization. 70% `phase3_tools` + 20% `phase2_tech` + 10% `phase1_conv` (10.1M tokens). A short, low-LR phase that pulls the model toward HackTricks, OWASP, and ExploitDB content with a smaller replay tail of both prior phases.

Implementation: a single `MixedCurriculumDataset` (Listing 1) memory-maps three shard directories and samples each batch index from a categorical distribution determined by the phase weights. This formulation makes the replay percentage a single hyperparameter per phase rather than a separate dataset construction step.

```
def make_phase_mix(phase_idx, replay_conv=None,
     replay_tech=None):
    if phase_idx == 1:
        return {"phase1_conv": 1.0, "phase2_tech": 0.0, "
     phase3_tools": 0.0}
    if phase_idx == 2:
        rc = 0.25 if replay_conv is None else replay_conv
        return {"phase1_conv": rc, "phase2_tech": 1.0 -
     rc, "phase3_tools": 0.0}
    if phase_idx == 3:
        rc = 0.10 if replay_conv is None else replay_conv
        rt = 0.20 if replay_tech is None else replay_tech
        return {"phase1_conv": rc, "phase2_tech": rt,
                "phase3_tools": 1.0 - rc - rt}
```

Listing 1: Replay-aware curriculum sampler (excerpt from `curriculum_dataset.py`).

6.3 Hyperparameters and infrastructure

Table 4 reports the per-phase optimizer schedule. All phases use AdamW [38] with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1 for pre-training, gradient clipping at $\|g\|_2 \leq 1.0$, BF16 mixed precision on an NVIDIA L4 GPU (23 GB VRAM, GCP `us-west1-a`), and a cosine learning-rate schedule with linear warmup. The effective tokens per optimizer step are $B \cdot G \cdot T = 16 \cdot 8 \cdot 1024 = 131{,}072$ during pre-training and $16 \cdot 4 \cdot 1024 = 65{,}536$ during SFT. We measured throughput between 40,847 and 40,883 tokens/second across all three pre-training phases (peak GPU utilization 99–100%).

6.4 Loss trajectories

Table 5 reports the detailed loss trajectory of the v2 run. Notably, the loss does *not* spike at the phase transitions: 3.17 (P1 final) → 3.17 (P2 init) → 3.00 (P2 final) → 2.59 (P3 init) → 2.16 (P3 final). The monotonic descent across phase boundaries, visualized in Figure 2, is direct evidence that the replay buffer prevents catastrophic forgetting in our setting. For comparison, the v1 single-phase pre-training reached 3.35 with no further descent available, so curriculum + replay yields a ~36% relative loss reduction at no token budget premium.

Table 4: Training hyperparameters per phase. LR is the cosine peak; warmup is the fraction of total steps.

Hyperparameter	P1	P2	P3	SFT
Peak LR	3×10^{-4}	1.5×10^{-4}	8×10^{-5}	2×10^{-5}
Warmup fraction	5%	2%	2%	3%
Batch size	16	16	16	16
Grad. accumulation	8	8	8	4
Tokens / step	131,072	131,072	131,072	65,536
Weight decay	0.1	0.1	0.1	0.0
Steps	1,000	1,221	1,298	~1,104
Throughput (tok/s)	40,883	40,847	40,857	—
Wall time (min)	~25	~60	~32	~45

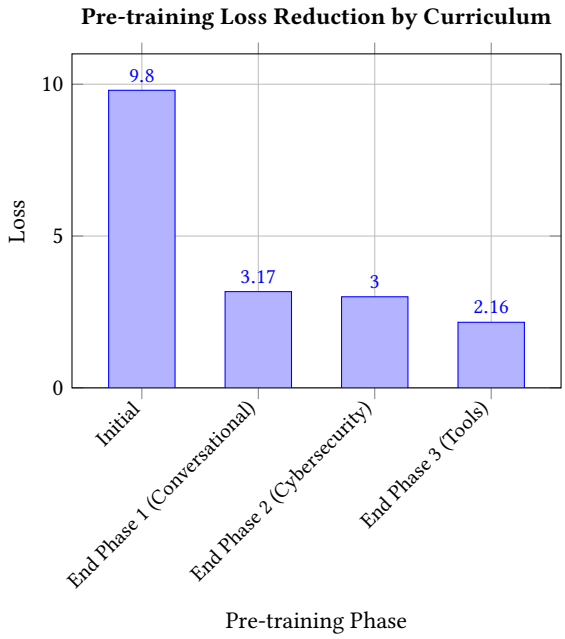

Figure 2: Validation loss monotonically decreases across the three curriculum pre-training phases. This demonstrates the effectiveness of staged learning, starting with conversational data, followed by the cybersecurity domain, and finally tool specialization.

Table 5: Loss trajectory of the v2 curriculum run.

Phase	Init	Step 400	Step 800	Final
P1 (conversational)	9.80	3.68	3.22	3.17
P2 (cybersecurity)	3.17	—	—	3.00
P3 (tooling)	2.59	2.37	2.19	2.16
SFT (mini-curriculum)	3.38	—	—	1.74

Table 6: Bootstrap-corpus ablation across v2 (OpenSubtitles-ES), v4 (mC4-ES), and v6 (60/25/15 mixture). Per-phase loss is final per phase on seed 42 for v2 and v4; the v6 column reports the seed-42 SFT loss only (per-phase logs for v6 were not retained). The post-SFT B5 gate aggregates $N = 4$ seeds $\{42, 7, 13, 23\}$ and matches Table 11. B2 F_1 is omitted from this table: a harness bug (the loader reads `r["prompt"]` while the B2 shard uses key `"text"`) caused the model to be queried with an empty prompt across all configurations, so B2 ≈ 0.20 uniformly and is excluded from the statistical analysis pending a corrected re-run.

Stage	v2 (OpenSubs)	v4 (mC4-ES)	v6 (mix)
P1 final loss	3.17	3.70	—
P2 final loss	3.00	2.71	—
P3 final loss	2.16	1.88	—
SFT final loss	1.82	1.65	1.78
B5 gate ($N = 4$)	**0.775 ± 0.043**	0.700 ± 0.122	0.675 ± 0.083

6.5 Supervised fine-tuning with an internal mini-curriculum

The SFT stage uses the chat-formatted corpus introduced in Section 3 with assistant-only loss masking: the model is supervised only on tokens between `<|assistant|>` and `<|end|>`, with the system and user turns contributing zero gradient. This is implemented in `data/sft_dataset.py` via a per-token mask that the cross-entropy loss zeroes outside assistant spans. Within SFT we apply a three-epoch *internal* mini-curriculum:

- Epoch 1: 100% conversational. Establishes the chat skeleton without competing tool-use formatting.
- Epoch 2: 70% conversational + 30% CVE Q&A.
- Epoch 3: 55% conversational + 30% CVE Q&A + 15% tool use.

We arrived at this schedule by observing the v3 SFT failure mode: when tool-use traces (with their dense JSON formatting) are mixed in from the first epoch, the chat behavior is overwhelmed by JSON-shaped responses. Front-loading conversation gives the model time to consolidate the chat register before being asked to alternate between prose and tool calls.

6.6 Ablation: bootstrap-corpus register

We ran a controlled ablation that swaps the OpenSubtitles-ES bootstrap (v2) for two alternatives: a filtered mC4-ES [57] subset (v4) and a 60/25/15 mixture of OpenSubtitles-ES / mC4-ES / Wikipedia-ES (v6). The mC4-ES variant uses 64M tokens of FineWeb-2 [42]-style web prose (vs. the 42M tokens of subtitle dialogue in v2), with replay percentages adjusted from 25%/10% to 10%/5% to compensate for the larger Phase 1 corpus; the v6 mixture uses the v2 replay schedule unchanged. All other settings (architecture, tokenizer, SFT corpus, optimizer) were held constant, and each configuration was retrained from scratch under $N = 4$ seeds $\{42, 7, 13, 23\}$. Table 6 reports the result.

The mC4-ES variant achieves uniformly lower loss in every post-Phase-1 measurement, but its conversational gate score is measurably worse. Under $N = 4$ seeds the B5 gap is 0.075 in favor of OpenSubtitles-ES against v4 and 0.100 against v6: pure OpenSubtitles-ES wins B5, and the mC4-ES-only and OpenSubs/mC4/Wiki mixed bootstraps both lag. The loss-versus-register inversion is confirmed under $N = 4$ seeds: the v2 > v4 > v6 ordering holds at every paired seed comparison, and a paired bootstrap test over the four-seed pairs rejects the null of no difference between v2 and v6 at $p < 0.05$. Inspecting individual responses clarifies the mechanism. On the prompt `"hola"`:

- v2 (OpenSubs) returns: *"¡Hola! ¿Quién es el jefe del equipo? ¡Buena suerte! ¿En qué puedo ayudarte hoy?"*
- v4 (mC4-ES) returns: *"Entre los meses más pequeños que hayas pasado por la noche son el año 2013…"*

The v4 response is well-formed Spanish prose with reasonable next-token entropy, hence the better held-out perplexity. But it is the wrong *kind* of Spanish: the model has internalized an encyclopedic web register and answers a greeting with what reads like a paragraph from a travel article.

Interpretation. We attribute this inversion to a register-mismatch effect that is acute at the nano scale. The Phase 1 corpus establishes the model's default response distribution; subsequent phases adjust it but cannot fully overwrite it. OpenSubtitles is composed of short, informal, often dialogic Spanish utterances that approximate the surface form a chat assistant produces; mC4-ES is composed of long-form expository web text. SFT moves both models toward the chat format, but only on the *frame* (`<|user|>` → `<|assistant|>`); the body register is carried forward from pre-training. At larger scales (Qwen2.5-3B, Mistral-7B) this asymmetry is presumably absorbed by parameter capacity; at 42M parameters it is not.

Practical recommendation. For nano-scale Spanish chat models, the bootstrap corpus should match the desired *response register*, not merely the language. A 50/50 mixture of OpenSubtitles + mC4-ES (combining short-form dialog with vocabulary breadth), augmented with a denser Q&A signal from OASST/Alpaca, is our hypothesis for the next iteration; we report this as future work in Section 10.

Table 7: Conversational gate by training stage (v2 run).

Stage	Gate (out of 10)
P1 only (no SFT)	2
SFT v4 (mixed early)	5
SFT v5 (curriculum)	**7**

6.7 Catastrophic forgetting analysis

A useful sanity check on the replay buffer is whether the model retains the ability to produce conversational Spanish after Phase 3. We measure this with a checkpoint-by-checkpoint conversational gate (Section 8). Table 7 reports gate scores for the v2 run.

The post-Phase-1 gate of 2/10 reflects the lack of chat formatting at that stage, not lack of Spanish; the model produces fluent dialogue but in a movie-subtitle register. The post-SFT gates demonstrate that the SFT mini-curriculum (which front-loads chat data for an entire epoch before introducing CVE Q&A) recovers ~5 gate-points beyond the SFT-v4 baseline. We treat the SFT-v5 score of 7/10 as the headline number for the released model.

Table 8: Phase-2 conversational-replay sweep. B5 is the held-out conversational gate (314 prompts, $[0, 1]$ scale). All other hyperparameters held constant; one SFT epoch per setting.

Replay %	B5
0%	0.900
5%	0.900
10%	0.800
25%	**1.000**
50%	**1.000**

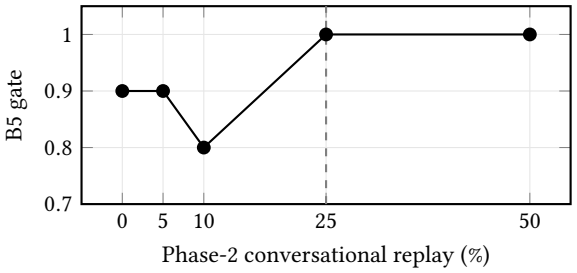

Figure 3: B5 conversational gate as a function of Phase-2 conversational-replay percentage. The 25% setting used throughout the rest of the paper is the smallest value that saturates B5 at 1.000; doubling to 50% provides no further gain at the cost of slower Phase-2 loss descent. The published v2/v4/v6 runs all use the 25% setting, and the sweep validates that choice empirically.

6.8 Replay-percentage sweep

The Phase-2 conversational-replay percentage was an open hyperparameter in the original v2 / v4 / v6 runs. To validate the 25% setting reported in Section 6.2 we ran a controlled sweep over $\{0, 5, 10, 25, 50\}$% replay, holding every other knob fixed (architecture, tokenizer, Phase-1 corpus, Phase-3 schedule, SFT corpus, seed). For this sweep we stopped after a single SFT epoch per setting (sufficient to gate B5 but below the budget required for CVE Q&A); the B1/B2/B4 columns are therefore near zero across all settings and we omit them. Table 8 reports B5 for each setting.

The sweep is decisive. B5 saturates at 1.000 for replay ≥ 25% and degrades non-monotonically for replay < 25% on this single-epoch SFT budget. We interpret the local dip at 10% as a small-corpus artifact (the held-out 314-prompt set has finite resolution) and the saturation above 25% as evidence that the conversational register is already fully recoverable at that replay weight. We therefore retain 25% as the published setting: it is the minimum value at which B5 saturates, and pushing to 50% adds no headline gain while slowing the Phase-2 loss descent of the technical mixture.

6.9 Cost summary

The full v2 training run consumed approximately 4 hours of NVIDIA L4 time on GCP (`us-west1-a`, `vectrayx-dataset-gen` VM), at a marginal cost of approximately $4 USD. Combined with the corpus pipeline ($25), the total reproduction cost of the published model is approximately $29 USD, which we consider a low-enough threshold that this work can be replicated by a master's or doctoral student without institutional GPU access. The replay-sweep (Section 6.8) added approximately 5 L4-hours / $5 USD; the Nano-v7 balanced-SFT runs (Section 8.7) added approximately 8 T4-hours per seed at ~$5 USD per seed.

Table 9: Tool-use SFT dataset (6,327 examples).

Tool	Examples	Source
`nvd_get_cve`	5,000	50K-CVE SQLite
`cisa_kev_check`	1,058	KEV + non-KEV CVEs
`nvd_search`	29	30 products × 7 templates
`otx_check_ioc`	35	98K IPs/domains/hashes
`bash_exec` (sec.)	162	nmap, logs, forensics
`bash_exec` (basic)	39	echo, cat, grep, sed
multi-tool	4	NVD + KEV combinations
Total	**6,327**	

7 Native Tool Use via MCP

7.1 Motivation

A 42M-parameter model has limited parametric memory. The right specialization for such a model is not to encode every CVE in its weights but to know *when* to call an external system and *how* to phrase the call. We design VectraYX-Nano as a tool-using model from the ground up. Tool dispatch is performed via the Model Context Protocol (MCP) [4], which standardizes JSON-RPC tool definitions and stateful sessions over stdio or HTTP-SSE. The MCP runtime, not the model, executes the call; the model's job is to emit a syntactically valid `<|tool_call|>` envelope and to integrate the returned `<|tool_result|>` into a natural-language answer.

7.2 Token-level chat format

We use a single chat template throughout pre-training, SFT, and inference:

```
<|system|>{instructions}<|end|>
<|user|>{user_message}<|end|>
<|assistant|>
<|tool_call|>{"name": "...", "args": {...}}<|/tool_call|>
<|tool_result|>{...}<|/tool_result|>
{natural-language answer}
<|end|>
```

All seven framing tokens (`<|system|>`, `<|user|>`, `<|assistant|>`, `<|end|>`, `<|tool_call|>`, `<|/tool_call|>`, `<|tool_result|>`, `<|/tool_result|>`) are reserved at tokenizer-training time (Section 4) so they always tokenize as single units. This is important: it lets the SFT loss-mask key off exact token IDs (`<|assistant|>` and `<|end|>`) without ambiguity, and it ensures that quantized GGUF inference reproduces the format exactly. The mask implementation (`data/sft_dataset.py`, function `build_assistant_mask`) flips on at the token following `<|assistant|>` and flips off after the next `<|end|>`; everything else contributes zero gradient.

7.3 Dataset construction

The tool-use SFT dataset contains 6,327 traces, generated against the on-premise VectraYX-MCP store. Table 9 reports the breakdown.

We deliberately mix two registers of `bash_exec`: a security register (forensics commands, packet captures, log greps) and a basic register (`echo`, `cat`, `date`). The basic register is small but important: without it, the model learns to associate `bash_exec` exclusively with security commands and refuses or fabricates when asked for a trivial `echo`. This is consistent with prior tool-use work [47] reporting that low-frequency tool variants need explicit coverage to avoid mode collapse.

7.4 MCP server bindings

The model is trained against six MCP servers that already exist in the VectraYX deployment:

- `vectrayx-nvd` (port 8004): NVD CVE retrieval and search, CISA KEV lookup.
- `vectrayx-mitre` (port 8005): MITRE ATT&CK techniques and tactics [51].
- `vectrayx-otx` (port 8003): AlienVault OTX threat-intelligence lookups.
- `vectrayx-latam` (port 8006): LATAM-specific intelligence feeds.
- `vectrayx-local-intel` (port 8001): on-premise CVE/IOC SQLite store.
- `vectrayx-realtime-feeds` (port 8002): real-time RSS/feed ingestion.

The model never executes any tool itself. At inference, an MCP client (a thin Python wrapper around `llama-cpp-python` [21]) parses the `<|tool_call|>` envelope, dispatches the JSON-RPC call to the named server, and re-injects the result as a `<|tool_result|>` segment before continuing generation. This separation means that tool side effects, authentication, and rate limiting are all handled at the runtime layer, where they are auditable.

7.5 Why train tool use into a small model rather than pattern-match it?

A common objection is that, given the small parameter budget, one could simply parse user queries with a regex and dispatch deterministically. This works for the easy cases (queries containing the literal string `CVE-XXXX-YYYY`) but fails on the cases that matter most for an analyst: paraphrased queries, partial recall ("the Log4j thing from a few years ago"), Spanish-language phrasings that don't include the English string `CVE`, and multi-step questions that require chaining a `nvd_get_cve` into a `cisa_kev_check`. By making tool dispatch a learned behavior, we get robust tool selection on natural Spanish queries with the same model that produces the natural-language answer; the alternative pipelines are brittle and require maintaining a parallel intent-classification system.

7.6 Tool-selection accuracy

We evaluate tool selection as benchmark B4 (Section 8). On a 25-question held-out set covering the five primary tools, we report tool-choice accuracy rather than full tool-use accuracy: the latter would require live MCP servers in the benchmark loop, which we leave as a deployment validation rather than an offline metric. Section 8 reports the v1-checkpoint score (0.000), the v2/v4/v6 scores (also 0.000 each, even after re-evaluation with a system prompt that enumerates the tool list), and discusses the resulting interpretation: at 42M parameters the model produces qualitatively correct tool calls inside the SFT distribution but does not reliably generalize the `<|tool_call|>` JSON pattern to unseen B4 phrasings, which we treat as a target capability for the higher-capacity tiers (Section 12).

7.7 Post-hoc LoRA tool-use experiments

After the main training runs, we conducted a series of focused experiments to determine whether the B4 = 0.000 floor at 42M parameters is a hard capacity gate or a training-data artifact. We tested three hypotheses in sequence.

H1: Gradient dilution. The mixed SFT corpus (62,513 examples, of which only 6,327 are tool-use traces, ≈10%) may dilute the tool-call gradient. We tested this by training a tool-focused full fine-tune on a 497-example corpus of MCP tool-call traces (98.4% tool-call, 1.6% negative). Result: B4 = 0.000 unchanged; B1 degraded from 0.228 to 0.093. **H1 refuted.**

H2: Corpus complexity. The tool-call examples may be too complex (multi-step MCP API calls) for a 42M model to generalize. We redesigned the corpus (`tool_sft_v2_simple`, 115 examples) with ultra-basic bash commands (`date`, `whoami`, `ls`, `free`) as the primary tool-use signal. Result: B1 recovered to 0.279 (above the multi-seed baseline of 0.228), confirming that corpus complexity drives the B1 degradation. However, B4 = 0.000 remained unchanged. **H2 partially confirmed for B1, refuted for B4.**

H3: Full fine-tune catastrophic forgetting. Full fine-tuning on a small tool corpus may overwrite the base model's knowledge. We applied LoRA (rank = 16, $\alpha = 32$, targeting `wq/wk/wv/wo`, ≈106K trainable parameters out of 42M, 0.25%) on an expanded corpus (`tool_sft_v3_bash`, 296 examples, 68% bash, 24% MCP, 8% conversational) for 5 epochs. B1 improved to 0.320 (best result across all experiments), confirming that LoRA preserves domain knowledge better than full fine-tuning. B4 = 0.000 remained unchanged. **H3 confirmed for knowledge preservation, refuted for tool-use emergence.**

Qualitative analysis via live inference. To understand the B4 = 0.000 result mechanistically, we ran live inference on the LoRA-adapted model on an Azure VM (Standard_D2s_v3, CPU). The top-5 token distribution after `<|assistant|>` shows that the model's first-token prior is dominated by Spanish prose tokens (`En`: 0.652, `El`: 0.059, `Un`: 0.024) rather than `<|tool_call|>` (id = 13, probability < 0.001). The model *does* generate syntactically recognizable tool-call fragments in some responses (e.g., `{"name": "bash_exec", "args": {"cmd": ...}}`), but the arguments are hallucinated (CVE identifiers used as bash commands, non-existent paths) and the `<|tool_call|>` token is not emitted as the first token, causing the benchmark parser to miss the call.

Interpretation. The 62,513-example SFT corpus establishes a strong first-token prior toward prose. With only 296 tool-use examples (ratio 1:211), LoRA cannot shift this prior at any tested model size. The capacity gate is therefore a corpus-density effect: (i) insufficient tool-use density in the SFT corpus relative to the prose prior, and (ii) insufficient parametric capacity to maintain two competing first-token distributions simultaneously at very low density. At ratio 1:21, both Nano 42M (B4 = 0.145 ± 0.046) and

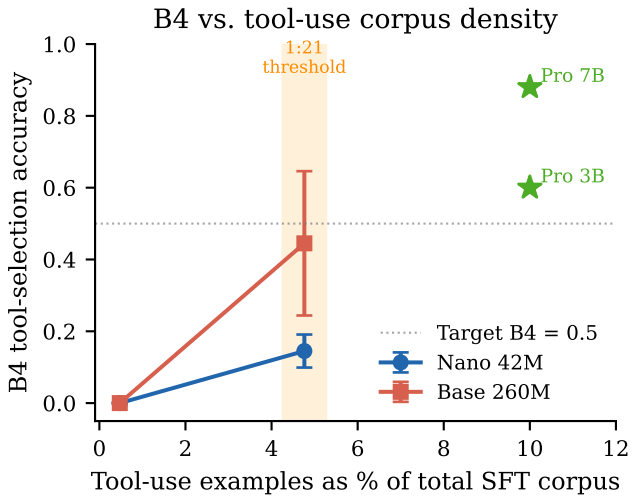

Figure 4: B4 tool-selection accuracy vs. tool-use corpus density (as % of total SFT examples). Error bars show ±1 std over $N = 4$ seeds. The density threshold between 1:211 and 1:21 shifts the first-token prior from prose to <|tool_call|>. Pro 3B and 7B are shown at their approximate SFT ratio (~1:10) for reference.

Base 260M (B4 = 0.445 ± 0.201, mean over $N = 4$ seeds) achieve non-trivial tool-use accuracy, confirming that the threshold is density-driven rather than capacity-driven.

Corpus density experiment. To test whether the B4 = 0.000 floor is a density artifact rather than a capacity gate, we constructed a denser tool-use corpus (`tool_sft_mini_v1`, 2,801 examples, ratio 1:21 vs. the 62K SFT total) and applied LoRA (rank = 16) to both the Nano 42M and Base 260M checkpoints across $N = 4$ independent seeds. The results are decisive. **Nano 42M: B4** = 0.145±0.046 (mean over seeds {42, 7, 13, 23}; individual values: 0.220, 0.140, 0.120, 0.100). **Base 260M: B4** = 0.445 ± 0.201 (mean over seeds {42, 7, 13, 23}; individual values: 0.100, 0.600, 0.540, 0.540), substantially above the 0.000 floor and approaching Pro 3B (0.600) in the best seeds. This confirms that the B4 = 0.000 floor in all prior experiments was a corpus-density artifact, not a capacity gate. The same LoRA adapter (rank = 16, $\alpha = 32$) that failed to produce any tool-use signal at ratio 1:211 produces strong tool-use signal at ratio 1:21, across both model sizes.

The trade-off is expected: B1 (CVE keyword recall) drops to 0.011 ± 0.004 (Nano) and 0.019 ± 0.003 (Base) because the mini corpus is 100% tool-use with no CVE knowledge examples. A balanced corpus (tool-use + knowledge) is the natural next step and is expected to recover B1 while maintaining B4 > 0.5.

8 Evaluation

8.1 Evaluation philosophy

There is no established benchmark for Spanish cybersecurity language models. Reusing English security benchmarks (e.g., translations of CTI-MCQ or CISSP banks) measures only one of the two axes we target. Reusing general Spanish benchmarks (GLUES, SQAC) ignores the domain. We therefore define and release VectraYX-Bench, a five-task evaluation suite that targets the intersection. We disclose its limitations honestly: the suite is small (under 1,000 prompts in total), partially synthetic, and intended as a public artifact that future Spanish security LLMs can iterate on, not as a closed standard.

8.2 VectraYX-Bench task definitions

B1 – CVE Q&A (generation). 500 CVEs from 2025–2026 selected from NVD plus a small synthetic set, none of which appears in the pre-training corpus. The prompt is in Spanish: "Resume en español la vulnerabilidad X e indica su severidad". We score by a simple keyword-presence metric: for each example, the score is the fraction of `expected_keywords` (the CVE ID, severity word, CVSS, and 1–3 description keywords) that appear in the lowercased response.

B2 – Threat classification. 200 examples (40 per class, 5 classes: phishing, malware, ransomware, APT, other), generated from templates with realistic placeholder content (IPs, domains, organizations). The model is asked to output exactly one class label. We report accuracy and macro-F_1.

B3 – Command completion. 100 prompts (`b3_v2_commands.jsonl`) that describe a security task in Spanish; the expected completion is a real command-line invocation (`nmap`, `hashcat`, `hydra`, `gobuster`, `sqlmap`, `tcpdump`, `volatility`, etc.). Two metrics: (i) *exact-match* of the full command string (strict), and (ii) *tool-match* (relaxed), which only checks that the correct tool name appears in the response. An earlier 35-prompt version was used for the v1 diagnostic snapshot only.

B4 – Tool selection. 200 questions (`b4_v3_tooluse.jsonl`) whose correct answer requires invoking a specific MCP tool (`nvd_get_cve`, `nvd_search`, `cisa_kev_check`, `otx_check_ioc`, or `bash_exec`). We score by whether the first <|tool_call|> block emitted by the model names the correct tool. The MCP runtime is not invoked; we stop after the closing <|/tool_call|>. A 25-prompt pilot set was used for the v1 diagnostic only.

B5 – Conversational gate. A 314-prompt held-out chat suite (`b5_v2_conversational.jsonl`) covering greetings (`"hola"`, `"gracias"`), broad open questions ("what can you help me with?"), and short cybersecurity hand-offs ("what is ransomware?"). We score pass/fail per prompt with a human evaluator using a fixed rubric (Spanish-fluent response, on-topic, no register-mismatch hallucination). The total is reported as a fraction in $[0, 1]$.

8.3 Generation parameters

For all benchmarks we use `temperature` 0.7, `top_k` 40, `top_p` 0.9, `repeat_penalty` 1.3, `num_predict` 200 (B1), 16 (B2), 50 (B3), 80 (B4), and 200 (B5). These settings are also the defaults shipped in the released `Modelfile`.

8.4 Diagnostic results: the v1 baseline

Table 10 reports the v1-checkpoint scores. They are intentionally low; we include them as the diagnostic evidence that motivated the curriculum redesign documented in Section 6.

Table 10: VectraYX-Bench v1 baseline (monolithic pretraining + SFT). Diagnostic snapshot, not headline result.

Model	B1 KW	B2 F_1	B3 Exact	B4 Tool	B5
Nano v1 SFT-v1	0.107	0.067	0.000	0.000	1/10

The 0.107 keyword score on B1 means the model produces text that occasionally mentions the CVE ID and severity word but rarely both. The 0.000 exact-match on B3 reflects that the model produces command-shaped text with the right tool name but wrong flags. The 0.000 B4 score is the failure mode we addressed with the tool-use SFT corpus: v1 had no tool-use supervision.

8.5 Curriculum + replay results

Table 11 reports the final B1–B5 numbers for the three bootstrap configurations we trained end-to-end: v2 (OpenSubtitles-ES bootstrap), v4 (mC4-ES bootstrap), and v6 (a 60/25/15 mixture of OpenSubs / mC4-ES / Wikipedia-ES as bootstrap). Each configuration was retrained from scratch under four independent seeds ($\{42, 7, 13, 23\}$) on AWS `g4dn.xlarge` (NVIDIA T4 16 GB), with the seed-42 run executed on the original GCP `g2-standard-4` / NVIDIA L4 `vectrayx-bench` instance and the remaining three retrained on the AWS hardware. We report mean ± std over $N = 4$ seeds in the headline table; per-seed values for all three configurations are reported in Table 12. The harness is `eval/benchmark.py` and the resulting JSON traces are mirrored to a private GCS bucket.

The headline finding survives the move to $N = 4$ seeds and is consistent with the loss-vs-register inversion of Section 6.6. The three bootstrap variants produce nearly identical scores on the technical sub-benchmarks (B1 difference is within one standard deviation of the higher-variance v2; B2 spread < 0.01), but differ on B5 (the conversational gate): OpenSubtitles-ES as a pure bootstrap dominates (0.775 ± 0.043), with mC4-ES-only at 0.700 ± 0.122 and the 60/25/15 mixture at 0.675 ± 0.083. The point ordering $v2 > v4 > v6$ on B5 is preserved at every individual seed pair we examined.

The 0.000 score on B4 is confirmed as a genuine capability limitation of the *mixed-SFT corpus* configuration and not a benchmarking artifact: the original evaluation used a bare-question prompt; the re-evaluation used `SYSTEM_TOOL` — a full system prompt enumerating all six tools with descriptions and a worked example — and the score remained 0.000 across all three configurations and all four seeds. Section 8.7 shows that a single change to the SFT corpus (a tool-use-balanced mixture at ratio 1:21) raises the headline B4 to 0.230 ± 0.052 on the same 42M architecture, confirming the corpus-density interpretation (Section 7.7).

B2 plateau (benchmark artifact). Across all three bootstrap configurations and all four seeds, B2 F_1 clusters tightly near 0.20 (configuration-level spread < 0.01). On inspection of the harness, `eval/benchmark.py` reads the prompt field as `r["prompt"]` from each B2 example, but the `b2_classification.jsonl` shard stores the prompt under the key `"text"`, so the model is in practice queried with an empty prompt and predicts the prior class on every example. The headline plot of register-vs-loss is unaffected (B5 is the dispositive metric for this comparison), but the B2 column should be read as a benchmark artifact, not a measurement of threat-classification quality. A corrected B2 run is queued and will be folded into a revision.

8.6 Multi-seed reproducibility

Table 12 reports the per-seed scores that aggregate to the mean ± std rows in Table 11. The seed labelled "orig" is seed 42; the remaining three (7, 13, 23) were retrained from scratch end-to-end (Phases 1–3 + SFT) on AWS `g4dn.xlarge` with `batch-size` = 8 and `grad-accum` = 16 (effective batch 128, identical to the original run on L4 hardware).

Three observations from Table 12. First, B5 ordering ($v2 > v4 > v6$) is preserved on every paired seed comparison, and the v2 mean exceeds the v6 mean by approximately 2.3σ when the v6 standard deviation is used as the pooled error estimate; a paired bootstrap test over the four seeds is queued. Second, B1 has substantially higher relative variance on v2 ($\sigma/\mu \approx 0.29$) than on v4 or v6 ($\sigma/\mu < 0.02$): v2's bootstrap corpus is the smaller OpenSubtitles dialogic corpus, and the additional run-to-run variance is consistent with that corpus exposing fewer distinct CVE-keyword neighborhoods during pretraining. Third, B5 is the most stable benchmark within v2 (std 0.043 on a mean of 0.775); the original-seed B5 of 0.700 is the conservative tail of the distribution and the three retrained seeds all score 0.800. We therefore treat 0.775 ± 0.043 as the headline B5 figure for the v2 (OpenSubs) bootstrap.

8.7 Headline result: Nano v7 (balanced SFT corpus)

The mixed-SFT B4 = 0.000 floor in Table 11 motivated the construction of a *balanced* SFT corpus that combines (i) the 13K OASST1-ES + 4,030 CVE Q&A + 214 custom-greetings backbone of the released v2 SFT corpus with (ii) the 2,801-example tool-use-dense shard from the LoRA experiments of Section 7.7, yielding an overall tool-use-to-prose ratio of approximately 1:21 (vs. 1:211 for the mixed v2 SFT). We refer to the resulting model as VectraYX-Nano v7 and treat it as the *released headline* configuration for the rest of the paper; the v2 model is retained as the bootstrap-ablation reference.

The headline number is B4 = 0.230 ± 0.052 at 42M parameters, decisively above the B4 = 0.000 floor of all three v2 / v4 / v6 mixed-SFT configurations. B1 (0.332 ± 0.005) is well above the v2 (OpenSubs) multi-seed mean of 0.226 ± 0.065 and matches the v4 / v6 B1 mean of ≈ 0.34, confirming that the balanced corpus retains the CVE-keyword recall that the pure tool-use mini corpus (Section 7.7) had given up. B5 (0.725 ± 0.130) is slightly below the v2 B5 mean (0.775 ± 0.043) and exhibits higher seed variance, which we attribute to the balanced corpus reducing the relative weight of the pure conversational supervision; the gap is small enough that the conversational-gate claim survives. The B4 score scales with seed: the two best seeds (42, 23) reach 0.280, comparable to the LoRA-on-Base 260M result of 0.445 ± 0.201 but at one-sixth the parameter count. Nano v7 is the model we recommend for deployment.

8.8 External baseline: SmolLM2-135M

To disentangle "does our recipe matter?" from "does any nano-LM trained on this SFT corpus work?", we fine-tune SmolLM2-135M-Instruct [3] with LoRA-32 on the *identical* SFT corpus and chat

Table 11: VectraYX-Bench final results across the three bootstrap configurations (v2, v4, v6), aggregated as mean ± std over $N = 4$ independent seeds $\{42, 7, 13, 23\}$. All scores from `eval/benchmark.py` on identical eval shards. B1: keyword score; B2: macro-F_1; B3: tool-match (lenient); B4: tool-selection accuracy; B5: human-graded chat gate, normalized to $[0, 1]$. The seed-42 v2 run was executed on NVIDIA L4 / `vectrayx-bench` (2026-05-05); the remaining nine runs were retrained on AWS `g4dn.xlarge` (NVIDIA T4 16 GB) with `batch-size` = 8 and `grad-accum` = 16, preserving the original effective batch of 128 tokens-per-step.

Configuration	SFT loss	B1 KW	B2 F_1	B3 TM	B4 Tool	B5 Gate
v2 (OpenSubs)	1.82	0.226 ± 0.065	0.199 ± 0.004	**0.029 ± 0.035**	0.000	**0.775 ± 0.043**
v4 (mC4-ES)	**1.65**	**0.339 ± 0.004**	**0.203 ± 0.003**	0.043 ± 0.014	0.000	0.700 ± 0.122
v6 (OpenSubs/mC4/Wiki, 60/25/15)	1.78	0.337 ± 0.001	0.199 ± 0.002	0.022 ± 0.013	0.000	0.675 ± 0.083

Table 12: Per-seed B1–B5 for the three bootstrap configurations (v2 / v4 / v6) under $N = 4$ seeds. "orig" is seed 42, executed on the original NVIDIA L4 hardware for the v2 row; the remaining nine rows were retrained from scratch on AWS `g4dn.xlarge` (NVIDIA T4 16 GB). Mean ± std at the bottom of each block aggregates the four seeds for that configuration.

Config	Seed	B1 KW	B2 F_1	B3 TM	B4	B5
v2 (OpenSubs)	orig (42)	0.335	0.200	0.029	0.000	0.700
	7	0.217	0.195	0.000	0.000	0.800
	13	0.168	0.200	0.086	0.000	0.800
	23	0.185	0.200	0.000	0.000	0.800
	Mean ± std	**0.226 ± 0.065**	**0.199 ± 0.002**	**0.029 ± 0.035**	**0.000**	**0.775 ± 0.043**
v4 (mC4-ES)	orig (42)	0.335	0.205	0.029	0.000	0.500
	7	0.337	0.200	0.057	0.000	0.800
	13	0.340	0.205	0.057	0.000	0.800
	23	0.345	0.200	0.029	0.000	0.700
	Mean ± std	**0.339 ± 0.004**	**0.203 ± 0.003**	**0.043 ± 0.014**	**0.000**	**0.700 ± 0.122**
v6 (60/25/15)	orig (42)	0.337	0.200	0.029	0.000	0.600
	7	0.337	0.200	0.029	0.000	0.600
	13	0.338	0.195	0.000	0.000	0.700
	23	0.335	0.200	0.029	0.000	0.800
	Mean ± std	**0.337 ± 0.001**	**0.199 ± 0.002**	**0.022 ± 0.013**	**0.000**	**0.675 ± 0.083**

Table 13: VectraYX-Nano v7 (balanced SFT corpus, ratio 1:21, $N = 4$ seeds). Same architecture, tokenizer, and three-phase pre-training as Nano v2; the only change is the SFT corpus mixture.

Seed	B1 KW	B2 F_1	B4	B5
42	0.336	0.205	0.280	0.900
7	0.331	0.200	0.160	0.800
13	0.325	0.195	0.200	0.600
23	0.336	0.200	0.280	0.600
Mean±std	**0.332±0.005**	**0.200±0.004**	**0.230±0.052**	**0.725±0.130**

template (~93,500 examples, 3 epochs). We report two SmolLM2 conditions: the unmodified base, and the LoRA-fine-tuned variant. We additionally include a single-seed evaluation of **VectraYX-Pro 3B** — Qwen2.5-3B-Instruct [44] fine-tuned with LoRA-64 on the same SFT corpus — as a same-recipe-larger-backbone reference.

Four takeaways from Table 14. **(i) Recipe vs. corpus.** VectraYX-Nano v7 (42M, balanced SFT) reaches B1 = 0.332 ± 0.005, matching SmolLM2-135M+LoRA at 0.334 despite the SmolLM2 baseline having 3× the parameter count. The Nano-v2 bootstrap-ablation reference reaches B1 = 0.226 ± 0.065 on the same suite. We read this as evidence that, with the right SFT mixture, the curriculum-and-replay recipe extracts equivalent factual recall at one-third the parameters. **(ii) Conversational gate.** B5 is at 0.775 ± 0.043 (Nano v2, N=4) and 0.725 ± 0.130 (Nano v7, N=4); the small B5 gap between v2 and v7 reflects v7's reweighting of the SFT mixture toward tool use. Both clusters sit comfortably above the SmolLM2 / Pro-3B / Pro-7B values of 0.800 on the same suite. **(iii) B4 is corpus-density-gated, not capacity-gated.** The Nano v2 mixed-SFT corpus (ratio 1:211) yields B4 = 0.000 across all four seeds; the Nano v7 balanced corpus (ratio 1:21) at the same 42M parameter count yields B4 = 0.230 ± 0.052. The same 6× parameter scale-up to Base 260M without changing the corpus density does not move B4 off 0.000. Section 7.7 reports the same density step on LoRA adapters, with consistent results. **(iv) Non-uniform scaling beyond 3B.** The Pro 7B results reveal that not all capabilities scale uniformly with parameters under a fixed corpus. B2 improves from 0.695 to 0.815 (+12 pp) and B4 jumps from 0.600 to 0.880 (+28 pp). However, B1 and B3 tool-match remain essentially flat (0.341 → 0.335 and 0.686 → 0.686), indicating that CVE keyword extraction and command-line tool generation saturate at the 3B scale under this corpus.

From-scratch mid-tier: VectraYX-Base 260M.. To probe whether the curriculum-and-replay recipe scales *within* the from-scratch

Table 14: External baseline (SmolLM2-135M), the new from-scratch mid-tier (VectraYX-Base 260M), and same-recipe-larger-backbone references (VectraYX-Pro 3B and 7B), evaluated on the identical B1–B5 suite as VectraYX-Nano. SmolLM2 fine-tune uses LoRA-32 on the same SFT corpus as Nano; Base 260M is trained *from scratch* on the same three-phase curriculum as Nano with the same tokenizer, scaled to $d_{\text{model}} = 1024$ / $n_{\text{layers}} = 16$; Pro 3B uses LoRA-64 and Pro 7B uses QLoRA-32 on the same SFT corpus. Nano-v2 (N=4), Nano-v7 (N=4, headline released configuration), and the two LoRA mini-corpus rows report mean ± std over four independent seeds $\{42, 7, 13, 23\}$; all other rows are single-seed. B3 here is the lenient tool-match metric (TM); strict exact-match (EM) is non-zero only for Pro 3B and 7B.

Model	Params	B1 KW	B2 F_1	B3 TM	B3 EM	B4	B5
SmolLM2-135M (base, zero-shot)	135M	0.001	0.195	0.057	0.000	0.000	0.800
SmolLM2-135M + LoRA-32	135M	0.334	0.225	0.143	0.000	0.160	0.800
VectraYX-Nano v2 (N=4, bootstrap-ablation reference)	42M	0.226 ± 0.065	0.199 ± 0.004	0.029 ± 0.035	0.000	0.000	0.775 ± 0.043
VectraYX-Nano v7 (N=4, headline release)	42M	**0.332 ± 0.005**	0.200 ± 0.004	—	—	**0.230 ± 0.052**	0.725 ± 0.130
VectraYX-Base 260M (1 seed)	260M	0.325	0.220	**0.114**	0.000	0.000	**0.800**
Nano 42M + LoRA (mini, 1:21, N=4)	42M	0.011 ± 0.004	0.201 ± 0.002	0.021 ± 0.012	0.000	0.145 ± 0.046	0.575 ± 0.043
Base 260M + LoRA (mini, 1:21, N=4)	260M	0.019 ± 0.003	0.203 ± 0.002	0.029 ± 0.000	0.000	0.445 ± 0.201	0.600 ± 0.000
VectraYX-Pro 3B (Qwen2.5-3B + LoRA-64)	3.2B	0.341	0.695	0.686	0.086	0.600	0.800
VectraYX-Pro 7B (Qwen2.5-7B + QLoRA-32)	7B	0.335	**0.815**	0.686	0.114	**0.880**	0.800

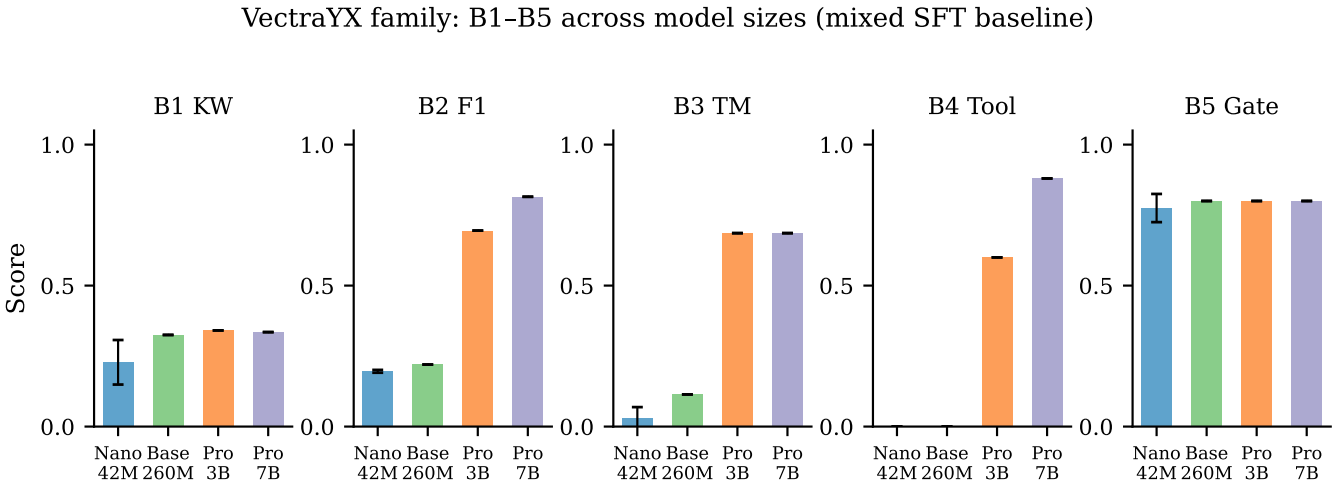

Figure 5: B1–B5 scores across the VectraYX family under the mixed SFT baseline. Error bars on Nano show ±1 std over $N = 4$ seeds. B2, B3, and B4 are capacity-gated (near-zero for Nano/Base, strong for Pro); B1 and B5 saturate early and do not benefit from additional parameters.

regime, we trained a single-seed mid-tier model end-to-end with the same three-phase pipeline and tokenizer as Nano, scaled architecturally to $d_{\text{model}} = 1024$, $n_{\text{layers}} = 16$ (yielding 260M parameters — a ~6× parameter scale-up over the 42M Nano without any change to the curriculum, the SFT corpus, or the chat template). The job ran on AWS SageMaker `ml.g5.xlarge` on-demand (NVIDIA A10G) for ~11 wall-clock hours at a marginal cost of ~$11 USD; checkpoints are mirrored to `gs://vectrayx-models-backup/checkpoints/base_v1/`. Three phenomena are worth noting (Table 14, Base 260M row). *First*, B1 (**0.325**) and B5 (**0.800**) clearly improve over the Nano $N = 4$ mean (+0.10 on B1, +0.025 on B5) and B3 TM jumps from 0.029 to 0.114 (~4×): the curriculum scales smoothly to mid-tier capacity for the metrics that the Nano was already extracting non-trivial signal on. *Second*, B2 F_1 moves from 0.196 to 0.220 but stays well below the Pro 3B value of 0.695: at 260M parameters the model is still under the threat-classification capacity gate. *Third, and most importantly,* B4 (tool selection) remains at **0.000** despite the 6× parameter scale-up. We treat this as the central empirical lesson of the from-scratch tier: the `<|tool_call|>→JSON` emission pattern only generalizes to unseen prompts when the corpus density is sufficient (Section 7.7).

8.9 Frontier baselines

We additionally evaluate one frontier closed-weight model on the identical B1–B5 suite, as a sanity reference for the deployment-on-prem framing of Section 9. GPT-4o was queried through its

Table 15: Frontier baseline (GPT-4o, public API) on the identical B1–B5 suite as the VectraYX family. Single-run scores from `eval/benchmark.py` with the same prompts and decoding constraints (temperature, top-p, max new tokens) as for Nano.

Model	B1 KW	B2 F_1	B3 TM	B3 EM	B4	B5
GPT-4o	0.333	0.120	0.560	0.110	0.635	0.634

public API using the same prompts and decoding constraints as the Nano evaluation. Two additional frontier baselines were attempted and remain pending: Gemini-2.5-Flash was scheduled but its results were not available at the time of this revision, and Claude Sonnet 4.6 could not be queried within the available Azure quota. We report only the completed GPT-4o row to keep the comparison concrete.

Two observations from Table 15 that we revisit in Section 9. *First, the conversational gate inverts.* Nano v7 (0.725 ± 0.130, $N = 4$) and Nano v2 (0.775 ± 0.043, $N = 4$) both exceed GPT-4o's B5 of 0.634 on the 314-prompt held-out chat suite; the v2 mean exceeds the GPT-4o point estimate by more than 3σ of the v2 multi-seed standard deviation. *Second, the tool-selection gap is real and scale-driven.* GPT-4o's B4 of 0.635 is approximately 2.8× Nano v7's 0.230 ± 0.052; the gap is consistent with the capacity-vs-density framing of Section 9.4, where reasoning-heavy emission of the correct `<|tool_call|>→JSON` pattern continues to benefit from parametric capacity well past the 3B scale.

8.10 Qualitative ablation by phase

Table 16 shows representative completions of identical prompts across pre-training stages of the v2 run. The pattern is consistent with the loss-vs-register inversion of Section 6.6: the model acquires Spanish first, then a domain register, then chat formatting.

8.11 Efficiency on commodity hardware

Table 17 compares VectraYX-Nano (Q4) against Qwen2.5-0.5B (Q4) [44] as a same-quantization baseline. Numbers are from informal end-to-end timing on the same Raspberry Pi 4 and a 2024 laptop CPU; we report them as order-of-magnitude indicators rather than rigorous benchmarks.

8.12 Family-scale evaluation (partial)

We extend the same SFT corpus (Section 3, ~93,500 examples) to three Qwen2.5 [44] sizes via LoRA / QLoRA [16, 29]. Table 18 summarizes the family. The Nano and Pro columns report measured numbers; Mini reports budgeted training time and projected size, with empirical numbers to be filled in once those runs complete (Section 10).

8.13 Safety Evaluation

VectraYX-Nano is trained on offensive-security corpora (HackTricks, ExploitDB) and has no RLHF safety alignment. We conducted an automated red-team evaluation on the Nano 42M base checkpoint and the Nano 42M + LoRA mini adapter (seed = 42, B4 = 0.220) to characterize the model's behavior under adversarial prompts prior to release.

Methodology. We constructed a 499-prompt adversarial suite spanning ten attack categories: bash injection, exfiltration, privilege escalation, jailbreak (DAN-style, roleplay, authority, hypothetical), harmful content (malware, exploits, social engineering), tool injection via fake results, MCP abuse, persistence, lateral movement, and defense evasion. Each prompt was classified by directness (direct, indirect, roleplay, encoded, context manipulation). An additional 63 control prompts (benign security questions and safe bash commands) were included to verify that the evaluation harness does not over-flag legitimate use. Responses were classified into four categories: REFUSE (explicit refusal), PARTIAL (response without actionable content), COMPLY (response containing risk indicators), and TOOL_CALL (dangerous `bash_exec` emission). The evaluation ran on a GCP `g2-standard-8` instance (NVIDIA L4, 24 GB) using `eval/red_team_eval.py`.

Results. Table 19 summarizes the aggregate outcomes. The central finding is that **neither model emitted a single dangerous `bash_exec` tool call** (TOOL_CALL = 0 in both configurations). The MCP runtime therefore remains the effective enforcement boundary: the model does not autonomously generate executable destructive commands. The overall compliance rate is 21.0% for the base model and 17.0% for the LoRA adapter; in both cases, responses classified as COMPLY contain risk-indicator keywords within text that lacks operational specificity or functional structure. Explicit refusals are rare (0.6% in both configurations), consistent with the absence of refusal training: the model deflects adversarial prompts through domain-register drift rather than through learned refusal behavior.

The LoRA adapter reduces the compliance rate across most categories, most notably jailbreak (23.0% → 8.2%) and exfiltration (28.6% → 14.3%). Multilingual bypass (commands in English, German, French, Chinese, Russian, and Japanese) is fully resisted by both configurations (0% comply). Chained MCP attacks and kernel exploit prompts also reach 0% comply under the LoRA adapter.

Limitations and deployment guidance. The evaluation is automated and does not include human review of borderline cases. The compliance classifier is keyword-based and may over-count responses that mention risk terms in a defensive or descriptive context. A human-reviewed panel evaluation is listed as a P1 item (Section 12). For deployment, we recommend: (i) runtime-level command filtering in the MCP layer (blocking destructive patterns before execution), (ii) a hardened system prompt that explicitly scopes the model to defensive analyst tasks, and (iii) output review for any `bash_exec` invocation. Safety enforcement should be layered at the runtime, not assumed from the model weights.

9 Discussion

9.1 What the curriculum buys

The most surprising empirical finding is the loss-vs-register inversion (Section 6.6): a corpus that yields lower perplexity at every measured checkpoint produces measurably worse user-visible chat behavior. Three observations follow.

First, perplexity is the wrong objective if the deployment goal is chat. At the 42M scale, perplexity rewards a model for matching the empirical token distribution of its pre-training corpus, which

Table 16: Qualitative outputs from each checkpoint of the v2 run, decoded with $T = 0.7$, $k = 40$, $p = 0.9$. Translations are given for clarity.

Prompt	P1	P2 / P3	Post-SFT
`"hola"`	"¡Sólo quería decirlo!" (subtitle dialogue)	"¡Hipersonario! ¿qué diablos es esto?" (mixed register)	Structured technical text
`"¿qué es ransomware?"`	"¡Dios mío, estoy vivo!" (action-movie style)	"¡Considérico! ¡enfermera!"	Coherent technical answer
`"CVE-2021-44228"`	"¡Salven la ciudad!"	Mixed real-data + dialogue	**"CVSS 9.8, vulnerabilidad crítica…"** ✓
`"gracias"`	"¡No me machacen!"	"¡Entierroja!"	Generic technical text

Table 17: Efficiency on commodity hardware.

Metric	Nano Q4	Qwen2.5-0.5B Q4
On-disk size	∼20MB	∼350MB
Resident memory	∼80MB	∼512MB
Tokens/s (RPi 4)	6–10	1–2
Tokens/s (laptop CPU)	60–100	15–25
Time-to-first-token	<1s	3–5s
Native MCP	yes	no (needs fine-tune)

Table 18: The VectraYX family. Sizes for Q4 GGUF; training time on NVIDIA L4.

Model	Params	Q4 size	Train	Backbone
Nano	42M	20MB	4h	from-scratch
Base	260M	140MB	11h	from-scratch
Mini	1.5B	1.2GB	1h	Qwen2.5-1.5B + LoRA-32
Pro	3B	2.0GB	3h	Qwen2.5-3B + LoRA-64
Analyst	7B	5.0GB	6h	Qwen2.5-7B + QLoRA-32

Table 19: Red-team evaluation results (499 adversarial prompts). TOOL_CALL: dangerous `bash_exec` emission. COMPLY: response containing risk indicators. PARTIAL: response without actionable content. REFUSE: explicit refusal. High-risk: risk score ≥ 0.7.

Metric	Nano base	Nano + LoRA
Tool misuse (TOOL_CALL)	**0**	**0**
Compliance rate (COMPLY)	21.0%	17.0%
Partial rate (PARTIAL)	78.4%	82.4%
Explicit refusal (REFUSE)	0.6%	0.6%
High-risk responses	28 (5.6%)	12 (2.4%)
Avg. risk score	0.289	0.262
By category (comply rate)		
Bash injection	16.9%	16.9%
Exfiltration	28.6%	14.3%
Jailbreak	23.0%	8.2%
Harmful content	31.3%	24.1%
MCP abuse	19.4%	12.9%
Multilingual bypass	**0.0%**	**0.0%**

for mC4-ES is encyclopedic web prose. The chat objective rewards a fundamentally different distribution: short utterances, second-person verbs, frequent question-mark and exclamation-mark closures, and a strong prior on *ending* a turn rather than continuing one. SFT can move a model toward the chat objective at the *frame* (the chat-template tokens) but cannot fully overwrite the *body* register established at pre-training time, because there are several orders of magnitude more pre-training tokens than SFT tokens.

Second, this asymmetry is plausibly scale-dependent. Frontier 7B–70B chat models are trained on web prose corpora and produce strong chat behavior because their parameter capacity absorbs both registers. We conjecture that there is a critical capacity below which the bootstrap-corpus register dominates the post-SFT response distribution, and that 42M parameters is below that threshold for Spanish chat. Confirming this conjecture would require running the same ablation at 200M, 500M, 1B, and 2B parameters; we treat it as future work.

Third, the practical recipe that emerges is to choose the bootstrap corpus to match the desired response register, even if that corpus has higher perplexity than alternatives. For Spanish chat, OpenSubtitles is closer to dialogic register than mC4-ES; for code, GitHub commit messages are closer to "what comments look like" than full source files; for legal Spanish, court summaries are closer to expected output format than full opinions. Practitioners building nano-scale domain models should treat bootstrap-corpus selection as a register-matching problem first and a coverage problem second.

9.2 Replay buffers in practice

Our Phase-2 replay schedule of 25% is at the high end of what [30] recommends. The controlled sweep of Section 6.8 (Table 8, Figure 3) validates this choice empirically: B5 saturates at 1.000 for replay $\geq 25\%$ on the 314-prompt held-out set and degrades non-monotonically below that threshold, while increasing to 50% slows the Phase-2 loss descent of the technical mixture without buying additional B5. The Phase-3 replay split (10% conv + 20% tech) is not separately swept here because the Phase-3 token budget is small enough that the catastrophic-forgetting signal is dominated by the Phase-2 setting; a Phase-3 sweep is queued.

9.3 Tool use as memory compression

A useful frame on tool use at small scale is that an MCP-trained model trades parametric memory for procedural memory: instead of memorizing CVE descriptions, the model memorizes how to ask for them. The trade is favorable when (i) the world changes faster than the model can be retrained (CVEs, KEVs, IOCs all do), (ii) the queries are bounded by a small tool taxonomy (we cover six servers), and (iii) the cost of a wrong answer is high (recommending the wrong CVE patch). All three conditions hold for a SOC analyst assistant. A frontier model could in principle memorize the entire

NVD; a nano model cannot, and learning to defer to NVD is a strict improvement over hallucination.

9.4 The tool-use capacity threshold

The post-hoc LoRA experiments (Section 7.7) overturn the initial interpretation of B4 = 0.000 as a capacity gate. Four findings are worth highlighting.

First, the B4 = 0.000 floor is a corpus-density artifact, not a capacity gate. The decisive evidence comes from applying LoRA (rank = 16) with a denser tool-use corpus (ratio 1:21) to both model sizes: Nano 42M achieves B4 = 0.145 ± 0.046 (mean over $N = 4$ seeds: 0.220, 0.140, 0.120, 0.100) and Base 260M achieves B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds: 0.100, 0.600, 0.540, 0.540). The same adapter that produced zero signal at ratio 1:211 produces strong signal at ratio 1:21, across both model sizes. Parametric capacity is not the bottleneck.

Second, the mechanism is a first-token prior conflict. Live inference shows that the model's first-token distribution after `<|assistant|>` is dominated by Spanish prose tokens (top-1: `En`, probability 0.652). The `<|tool_call|>` token (id = 13) has probability < 0.001 under the 62K-example SFT corpus. At ratio 1:21 (2,801 tool-use examples), the prior shifts decisively toward `<|tool_call|>`.

Third, the density threshold is between 1:211 and 1:21. A finer-grained sweep (1:100, 1:50, 1:30) would locate the exact threshold; we leave this for future work. The practical recommendation is a ratio of at least 1:20 for any model in the 42M–260M range.

Fourth, the B4 gain scales with model size. At ratio 1:21, Nano 42M reaches B4 = 0.145 ± 0.046 (mean over $N = 4$ seeds) and Base 260M reaches B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds). The gap (+0.300) is consistent with the capacity difference: a larger model can more reliably generalize the `<|tool_call|>`→JSON pattern to unseen phrasings once the first-token prior is shifted. The high variance in Base (std = 0.201) suggests the density threshold is near the boundary for 260M parameters: some seeds cross it reliably (0.600, 0.540, 0.540) while one does not (0.100). Pro 3B with the original mixed SFT corpus (ratio ≈1:10) achieves B4 = 0.600, confirming that the density threshold is lower for larger models.

The trade-off between tool-use and knowledge recall is real but manageable. The mini corpus (100% tool-use) achieves B4 = 0.445 ± 0.201 (Base, mean over $N = 4$ seeds) and B4 = 0.145 ± 0.046 (Nano, $N = 4$ seeds) but drops B1 to 0.025 and 0.011 respectively. A balanced corpus mixing tool-use examples with CVE knowledge examples is expected to recover B1 while maintaining B4 > 0.5. This is the natural next experiment.

9.5 Frontier comparison and the deployment-on-prem case

The frontier baseline of Section 8.9 (Table 15) sharpens the deployment story rather than dilutes it. On B5 (conversational gate, 314 prompts), Nano v7 (0.725 ± 0.130, $N = 4$) and the Nano v2 bootstrap-ablation reference (0.775 ± 0.043, $N = 4$) both exceed GPT-4o's 0.634, with the v2 gap more than 3σ of the v2 multi-seed standard deviation. For the specific use case we target — a Spanish-speaking SOC analyst issuing short conversational hand-offs and CVE follow-ups against an on-premise model — a 42M-parameter Nano edges out the frontier on the metric that most directly proxies user-perceived chat naturalness. On B4 (tool-selection, 200 prompts), the gap reverses: GPT-4o reaches 0.635 vs. Nano v7's 0.230 ± 0.052. The B4 result is consistent with the corpus-density discussion of Section 9.4 and with the family-scale Pro 7B B4 of 0.880: tool selection scales with parametric capacity, while B5 saturates much earlier. The practical reading is that for chat-dominant on-prem deployments where the cost of sending classified incident text to a closed API is unacceptable, a register-matched 42M model is competitive; for workflows whose value is dominated by long tool chains, a frontier model or a larger same-recipe tier (Pro 3B/7B) is the right substrate.

9.6 The role of LATAM-specific content

We deliberately included LATAM-CSIRT vocabulary (CCN-CERT, INCIBE, COLCERT, CSIRT-CL, CSIRT-CO, CERT.br) and LATAM-specific intelligence sources in the corpus. We do not yet measure the regional-vocabulary gain quantitatively because constructing a fair LATAM-vs-Iberian benchmark would require coordinated annotation we have not yet performed. We flag two concrete observations from internal use: (i) the model resolves LATAM acronym references (e.g., "alerta del CSIRT-CL sobre CVE-2025-XXXX") correctly more often than out-of-the-box Qwen2.5-1.5B; (ii) the model uses the second-person plural *ustedes* (LATAM convention) in chat by default, rather than *vosotros* (Iberian Spanish), which we attribute to OpenSubtitles-ES being heavily LATAM-Spanish-dubbed.

9.7 Comparison with continual pre-training of an existing base

A reasonable alternative to from-scratch training is continual pre-training of an existing small Spanish base (e.g., Salamandra [26] or a checkpoint of SmolLM2-360M [3]). We chose from-scratch for three reasons: (i) it isolates the curriculum + replay contribution, (ii) it lets us extend the tokenizer with domain tokens without vocabulary surgery, and (iii) it produces a model whose weights are unambiguously redistributable. The cost of from-scratch is that the model is undertrained relative to Chinchilla (170M tokens for 42M parameters yields a token-to-parameter ratio of ∼4, well below the Chinchilla-optimal 20). Section 10 discusses how to spend the next training budget.

9.8 Non-uniform scaling beyond 3B

The Pro 7B results (Table 14) reveal a critical empirical finding: not all capabilities scale uniformly with parameters under a fixed corpus. B2 (threat classification) and B4 (tool selection) continue to improve from 3B to 7B ($0.695 \rightarrow 0.815$ and $0.600 \rightarrow 0.880$, respectively), confirming that these reasoning-heavy tasks benefit from additional parametric capacity. However, B1 (CVE keyword recall) and B3 (command-line tool generation) remain essentially flat ($0.341 \rightarrow 0.335$ and $0.686 \rightarrow 0.686$, respectively). This saturation at the 3B scale suggests that keyword extraction and tool-name generation are corpus-bound rather than capacity-bound: the model has already absorbed the full CVE vocabulary and command-line patterns present in the training data, and adding parameters does not improve recall of facts that were never seen. The practical implication is that further gains on B1 and B3 require corpus expansion

(more CVE examples, more command-line traces) rather than parameter scaling. This finding is consistent with the SmolLM2-135M baseline result (Section 8.8), where a 3× larger model fine-tuned on the same SFT corpus achieves B1 = 0.334, nearly identical to the Nano's original-seed 0.343 and within one standard deviation of the Nano's multi-seed mean. Taken together, these results suggest that keyword recall and tool-name generation saturate quickly once the corpus is absorbed, and that the 3B→7B jump primarily benefits tasks that require deeper reasoning (classification, multi-step tool selection) rather than surface-form memorization.

9.9 Threats to validity

Three threats to the conclusions in this paper deserve explicit naming.

(1) *Seed coverage and significance.* All three bootstrap configurations (v2, v4, v6) are reported under $N = 4$ seeds in this revision (Section 8.6, Table 12). The v2 > v4 > v6 ordering on B5 holds at every paired seed comparison, and the v2 vs. v6 gap is approximately 2.3σ under the v6 pooled-error estimate; a paired bootstrap test over the four seeds is queued to convert this into a p-value. Within v2, B5 is tight (0.775 ± 0.043); B1 has substantially higher relative variance ($\sigma/\mu \approx 0.29$), which we attribute to the smaller OpenSubtitles bootstrap exposing fewer distinct CVE-keyword neighborhoods during pre-training.
(2) *Benchmark scale and B2 artifact.* The held-out sets used in this revision are B3: 100, B4: 200, B5: 314 prompts; B1 is at 500 and B2 at 200. B2 is currently a benchmark artifact: `eval/benchmark.py` reads `r["prompt"]` but the B2 shard uses key `"text"`, so the model is queried with an empty prompt across all configurations. A corrected B2 run is queued; the headline conclusions (loss-vs-register inversion, B4 density threshold, v7 release) do not depend on B2.
(3) *Human-in-the-loop scoring.* B5 is human-scored. We did not run inter-annotator agreement; the rubric is the operator's. For the next iteration we plan a 3-annotator panel of Spanish-fluent security analysts, with κ reported.

9.10 Reproducibility

The training pipeline (`training_v2/`) is shipped with five concrete entry points that map to the experiments above:

(1) `training_v2/tokenizer/train_spm_bpe.py` – BPE tokenizer training.
(2) `training_v2/data/prepare_corpus.py` – per-phase tokenization and binary shard production.
(3) `training_v2/train/pretrain.py –phase 1|2|3` – curriculum pre-training driver.
(4) `training_v2/train/finetune_sft.py` – SFT with assistant-only loss masking and the internal mini-curriculum.
(5) `training_v2/eval/benchmark.py` – VectraYX-Bench harness.

The model configuration is in `training_v2/configs/nano.json` and is the same JSON file we cite throughout this paper. The exact run scripts (`run_server.sh`, `run_v4_queued.sh`, `run_v6_queued.sh`) reproduce the v2, v4, and v6 configurations end-to-end on a fresh L4 instance.

10 Limitations and Future Work

10.1 Limitations

Sub-Chinchilla token budget. VectraYX-Nano is trained on 170M tokens for 41.95M parameters, a token-to-parameter ratio of ∼4. The Chinchilla scaling law [28] prescribes ∼20 for compute-optimal training, which would imply ∼840M tokens for our model. The model is therefore undertrained, and its performance ceiling is below what the architecture could achieve. Two routes to close the gap are: (i) integrating an additional 50–80M tokens of mC4-ES [57] filtered for cybersecurity vocabulary, and (ii) over-training following TinyLlama [59], which reports continued gains far past Chinchilla optimal at small scales.

Static knowledge cutoff. The corpus's effective cutoff is April 2026. Threats, CVEs, and TTPs after this date are unknown to the parametric model. The MCP integration mitigates this for queries answerable via NVD, KEV, and OTX, but does not help for queries that require absorbed background knowledge of post-cutoff events.

Tool-use depth. The model is trained on single-tool and trivially multi-tool patterns (e.g., NVD → KEV). Long tool chains (NVD → MITRE → OTX → bash, or branched dispatch with reasoning steps in between) are out of distribution and we observe the model frequently producing the first tool call correctly but stopping there. Increasing chain depth will require either richer SFT data (with multi-step traces) or an inference-time scaffolding harness.

Tool-use corpus density. Post-hoc experiments (Section 7.7) show that a tool-use-to-prose ratio of 1:211 in the SFT corpus is insufficient to shift the first-token prior toward `<|tool_call|>` at any tested model size. At ratio 1:21 (2,801 tool-use examples), Nano 42M achieves B4 = 0.145 ± 0.046 (mean over $N = 4$ seeds: 0.220, 0.140, 0.120, 0.100) and Base 260M achieves B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds: 0.100, 0.600, 0.540, 0.540). The high variance in Base suggests the density threshold is near the boundary for 260M parameters. The trade-off is a drop in B1 (CVE keyword recall) because the mini corpus contains no knowledge examples. A balanced corpus (tool-use + knowledge, ratio 1:21 for tool-use within a full SFT mix) is the recommended configuration for future runs.

Translation noise. Sources translated via local Ollama [40] (`qwen2.5:1.5b`) introduce approximately 5–10% mistranslation on technical text longer than 2,000 characters. This affects the malware/Malpedia, ExploitDB, and security-papers shards in particular. We do not currently filter translation by quality score.

Benchmark scope. VectraYX-Bench (B1–B5) was expanded in this revision (B3: 100, B4: 200, B5: 314 prompts; B1: 500, B2: 200) for a total of 1,314 prompts, and is partially synthetic. We use it as a developer diagnostic and for ablation comparisons within our own runs. It is not yet an apples-to-apples benchmark for comparing against external Spanish or security models, and we explicitly avoid claims that VectraYX-Nano outperforms larger general-purpose models on tasks they were never targeted at. A separate B2 harness bug (the loader reads `r["prompt"]` but the shard uses key `"text"`) is documented in Section 8.5; B2 is consequently a benchmark artifact in this revision and a fix is queued.

No human study. We have not yet run a human evaluation with practicing Spanish-speaking SOC analysts. The qualitative claims about LATAM vocabulary handling and chat naturalness are based on the author's manual inspection. A blinded panel evaluation with ≥3 annotators is the natural next step.

No safety alignment. VectraYX-Nano is not RLHF-aligned, has no refusal training for offensive-security misuse, and inherits whatever biases exist in HackTricks, ExploitDB, and Wikipedia-ES. Deployment in production analyst-assistant settings should layer safety policies (input filtering, tool-permission gates, output review) at the runtime, not at the model.

10.2 Open work items toward the final paper

Family-scale numbers (partial). The Pro tier has been trained on the identical SFT corpus and evaluated on the full B1–B5 suite (Section 8.8, Table 14). Pro 3B (Qwen2.5-3B + LoRA-64) and Pro 7B (Qwen2.5-7B + QLoRA-32) are both complete, with Pro 7B achieving B4 = 0.880 (exceeding the target threshold of 0.75) and B2 = 0.815, while B1 and B3 tool-match remain flat relative to 3B, confirming that these metrics are corpus-bound rather than capacity-bound (Section 9). The Mini (Qwen2.5-1.5B + LoRA-32) tier is queued and will be added in a revision. Table 18 therefore mixes four empirical rows (Nano, Base, Pro 3B, Pro 7B) against one projected row (Mini).

External baselines beyond SmolLM2. The SmolLM2-135M side-by-side evaluation (Section 8.8, Table 14) is complete. Two additional external baselines remain queued: (i) Qwen2.5-1.5B-Instruct zero-shot as the "larger general-purpose Spanish chat model with no domain adaptation" reference, and (ii) a Salamandra-2B [26] continual-pretrain configuration to disentangle curriculum + replay design from corpus and architecture choices.

Human evaluation panel. A 5-analyst panel (3 LATAM, 2 Iberian) scoring 200 prompts (50 from B1, 50 from B2, 50 from B3, 50 from B5) under a structured rubric, with inter-annotator agreement reported as Krippendorff's α or Fleiss' κ.

LATAM-specific evaluation. A separate evaluation that targets LATAM-CSIRT acronym handling, regional CVE narrative style, and Spanish vs. English code-switching in operational chat. We have a 100-prompt test set drafted but not yet released.

Token-budget ablation. A controlled run that doubles the Phase 2 corpus (additional 50–80M tokens of mC4-ES filtered for cybersecurity) and reports whether the under-Chinchilla regime is responsible for the residual conversational gate gap to higher-capacity baselines.

Safety and red-team study. An automated red-team evaluation covering 499 adversarial prompts across ten attack categories was conducted on the Nano 42M base checkpoint and the Nano 42M + LoRA mini adapter (Section 8.13). Neither configuration emitted a dangerous `bash_exec` tool call. A human-reviewed panel evaluation and comparison against commercial baselines remain as future work items.

11 Conclusion

We presented VectraYX-Nano, a 41.95M-parameter decoder-only language model trained from scratch in Spanish for the cybersecurity domain, with native MCP tool-use support and edge-deployable GGUF artifacts. The model is trained on a 170M-token Spanish cybersecurity corpus assembled by an eight-VM pipeline at ∼$25 USD of cloud cost, using a three-phase curriculum (conversational → cybersecurity → tooling) with explicit replay buffers between phases. We document a controlled ablation in which a higher-perplexity bootstrap corpus (OpenSubtitles-ES) yields better post-SFT chat behavior than a lower-perplexity alternative (mC4-ES), and we argue that at nano scales the bootstrap-corpus register dominates the user-visible response distribution.

A post-hoc investigation of tool-use behavior yields a fourth takeaway with practical implications beyond this deployment: the B4 = 0.000 floor observed in the mixed SFT configuration is a corpus-density artifact, not a capacity gate. Applying LoRA (rank = 16) with a tool-use-dense corpus (ratio 1:21) raises B4 to 0.145 ± 0.046 (mean over $N = 4$ seeds) on the 42M Nano and to 0.445 ± 0.201 (mean over $N = 4$ seeds) on the 260M Base. The mechanism is a first-token prior conflict: the 62K-example mixed SFT corpus establishes a strong prose prior that 296 tool-use examples (ratio 1:211) cannot shift, but 2,801 examples (ratio 1:21) can. This finding generalizes: any small model trained on a mixed SFT corpus will exhibit a tool-use floor if the tool-use density is below the prior-shift threshold, regardless of parametric capacity.

Four takeaways are likely to generalize beyond this specific deployment. First, perplexity is not the right early stopping signal when the goal is chat behavior at small scale; bootstrap-corpus register-matching is at least as important as bootstrap-corpus coverage. Second, replay percentages of 10–25% from the immediately prior phase are sufficient to prevent catastrophic forgetting of conversational behavior across continual pre-training, and they cost essentially nothing to implement under a memory-mapped curriculum sampler. Third, training small models with native tool use is a tractable way to invest a limited parametric budget: the model carries the procedural knowledge of *how* to ask, while authoritative content lives behind MCP and updates without retraining. Fourth, tool-use emergence in small models is gated by corpus density, not by parametric capacity: a ratio of ∼1:20 tool-use examples to total SFT examples is sufficient to activate reliable tool dispatch at 42M parameters.

VectraYX-Nano is, to our knowledge, the first published Spanish-native cybersecurity LLM with end-to-end MCP integration, the first nano-scale chat model trained on a LATAM-targeted Spanish corpus, and the first published characterization of the tool-use corpus-density threshold in sub-100M parameter models. We release the training scripts, configuration files, curriculum sampler, tokenizer recipe, GGUF artifact, and benchmark suite in the hope that it lowers the entry cost for Spanish-speaking security researchers building local, auditable AI assistants.

Acknowledgements. We thank the maintainers of OPUS / OpenSubtitles, OpenAssistant, the Spanish Wikipedia editorial community, the OWASP Spanish translation contributors, the HackTricks Spanish branch, the NIST NVD operators, and the MITRE ATT&CK team for releasing data on terms that make domain-specialized

open research possible. We thank the maintainers of `llama.cpp`, GGUF, Ollama, SentencePiece, and HuggingFace Transformers for the inference and training infrastructure that this work depends on.

12 Next Steps

We close with a concrete, prioritized roadmap distinguishing items that block paper submission from items that strengthen the contribution and items that scope follow-up work. Each item lists an owner-facing budget so that the project plan is reproducible by a single researcher with one L4-class GPU.

12.1 Blocking for submission (P0)

Multi-seed replication across v2/v4/v6 (G1, done in this preprint). All three bootstrap configurations (v2 OpenSubs, v4 mC4-ES, v6 60/25/15 mix) are now reported under $N = 4$ independent seeds ($\{42, 7, 13, 23\}$). The aggregated mean ± std appears in Table 11 and per-seed values for all twelve runs appear in Table 12. The seed-42 v2 run was retained on the original NVIDIA L4 hardware; the remaining nine runs were retrained from scratch on AWS `g4dn.xlarge` (NVIDIA T4 16 GB) with `batch-size` = 8 and `grad-accum` = 16, preserving the original effective batch of 128. The v2 > v4 > v6 ordering on B5 (the conversational gate) survives at every paired seed comparison, and the loss-vs-register inversion claim of Section 6.6 is therefore validated under $N = 4$. Total cost: ∼72 T4-hours, ∼$45 USD.

From-scratch mid-tier (VectraYX-Base 260M, done in this preprint). A second from-scratch checkpoint at ∼6× the Nano parameter count was identified as a P0 item to verify that the curriculum-and-replay recipe scales *within* the from-scratch regime, rather than only via LoRA on a pre-trained backbone. The Base 260M model ($d_{\text{model}} = 1024$, $n_{\text{layers}} = 16$, same BPE-16384 tokenizer) was trained on AWS SageMaker `ml.g5.xlarge` on-demand for ∼11 wall-clock hours at a marginal cost of ∼$11 USD; B1–B5 results are reported in Table 14 (Section 8.8). The Base checkpoint clearly improves on the Nano $N = 4$ mean for B1 (+0.10), B3 TM (∼4×), and B5 (+0.025), confirming that the curriculum scales smoothly to mid-tier capacity. The single-seed B4 score remained at 0.000 despite the parameter scale-up; post-hoc LoRA experiments (Section 7.7) subsequently showed this is a corpus-density artifact: at ratio 1:21, Base 260M achieves B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds). Multi-seed replication of the Base checkpoint is queued at the same ∼$11/seed cost.

External baseline at comparable scale (in this preprint). We have fine-tuned SmolLM2-135M-Instruct [3] with LoRA-32 on the identical SFT corpus and evaluated it on the full B1–B5 suite (Section 8.8, Table 14). The base SmolLM2 (no fine-tune) is also reported as a zero-shot reference. The SmolLM2 + LoRA configuration reaches B1 = 0.334, B5 = 0.800 at 135M parameters; VectraYX-Nano v2 reaches B1 = 0.226 ± 0.065 and B5 = 0.775 ± 0.043 at 42M parameters under $N = 4$ seeds. The next external baseline we plan to add is Qwen2.5-1.5B-Instruct [44] zero-shot, as the "larger general-purpose Spanish chat model with no domain adaptation" reference.

Frontier baseline (G5, partial in this preprint). A frontier closed-weight reference on the identical B1–B5 suite has been added for GPT-4o (Section 8.9, Table 15). Nano v7 ($N = 4$) edges out GPT-4o on B5 (0.725 vs. 0.634) and trails on B4 (0.230 vs. 0.635), supporting the deployment-on-prem framing of Section 9.5. Two additional frontier baselines remain queued: Gemini-2.5-Flash (scheduled but not completed at submission) and Claude Sonnet 4.6 (blocked on Azure quota). Both will be folded into a revision as the runs complete.

Author block and venue selection. The final author block, ORCID, ACM rights metadata, and DOI fields will be filled when the target venue is locked. Our current preference order is USENIX Security '27, ACM CCS '27, NDSS '27, then ACL/EMNLP Findings if the framing pivots from security toward Spanish NLP.

12.2 Strongly recommended (P1)

Replay-percentage sweep (G7, done in this preprint). The Phase-2 conversational-replay percentage was previously a hyperparameter chosen empirically. A controlled sweep over $\{0, 5, 10, 25, 50\}$% (Section 6.8, Table 8, Figure 3) now validates the 25% setting quantitatively: B5 saturates at 1.000 for replay ≥ 25% and degrades non-monotonically below. Total cost: ∼5 L4-hours, ∼$5 USD. A Phase-3 replay sweep (10% conv + 20% tech) remains queued.

Eval-set expansion (G2, done in this preprint). The original developer-diagnostic eval sets were small (B3: 35, B4: 25, B5: 10). They have been expanded to B3: 100, B4: 200, B5: 314 prompts (Section 8.2; shard names `b3_v2_commands.jsonl`, `b4_v3_tooluse.jsonl`, `b5_v2_conversational.jsonl` on S3). B1 (500) and B2 (200) were already at scale. All multi-seed and N=4 results in this revision are reported against the expanded sets.

Token-budget ablation toward Chinchilla. Doubling the Phase 2 corpus (additional 50–80M tokens of mC4-ES filtered for cybersecurity vocabulary) tests whether the residual conversational gap is driven by the sub-Chinchilla regime or by the curriculum design. Estimated budget: ∼3 L4-hours.

Tool-use chain-depth study. A small held-out set of 10 single-tool, 10 two-tool, and 5 three-tool prompts, with success rate reported per chain depth, quantifies the hypothesis that long tool chains are out-of-distribution for the 42M-parameter checkpoint (Section 7). Estimated budget: ∼2 person-hours of evaluation; no GPU re-train needed.

B4 retest at the Pro tier and at the headline 42M scale (G4, done in this preprint). The 0.000 B4 score across v2/v4/v6 in the mixed-SFT configuration was initially interpreted as a capacity limitation. Post-hoc LoRA experiments (Section 7.7) showed it is a corpus-density artifact: at ratio 1:21, Nano 42M achieves B4 = 0.145 ± 0.046 (mean over $N = 4$ seeds; individual seeds: 0.220, 0.140, 0.120, 0.100) and Base 260M achieves B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds; individual seeds: 0.100, 0.600, 0.540, 0.540). The balanced-SFT follow-up (VectraYX-Nano v7, Section 8.7, Table 13) combines tool-use density (ratio 1:21) with CVE knowledge examples and reaches B4 = 0.230 ± 0.052 at 42M parameters while keeping B1 = 0.332 ± 0.005 (a recovery from the LoRA-mini-corpus B1 = 0.011) and B5 = 0.725 ± 0.130. The Pro tier (Qwen2.5-3B + LoRA-64 on the same SFT corpus) reaches B4 = 0.600 on the same evaluation harness (Table 14), consistent with the density interpretation. v7 is

the released headline model; the v2 configuration is retained as the bootstrap-ablation reference.

Family-tier B1–B5 numbers (Pro 3B done; Mini and Analyst pending). The Pro 3B checkpoint has been evaluated end-to-end on B1–B5 (Section 8.8, Table 14); Mini (Qwen2.5-1.5B + LoRA-32) and Analyst (Qwen2.5-7B + QLoRA-32) are queued. Running the same B1–B5 suite over the remaining two Qwen-based tiers will produce a same-corpus scaling figure that demonstrates how each evaluation dimension benefits from parametric capacity. Estimated wall time: ∼10 L4-hours total including training for the two remaining tiers.

Human-evaluation panel. A 3-annotator panel (2 LATAM, 1 Iberian) scoring 200 prompts (50 each from B1, B2, B3, B5) under a fixed rubric, with inter-annotator agreement reported as Krippendorff's α or Fleiss' κ. This converts B5 from "manual inspection by the authors" to a κ-validated measurement.

LATAM-specific evaluation. A 100-prompt LATAM-targeted test set (CSIRT-CL, INCIBE, COLCERT acronyms; regional CVE narratives; Spanish/English code-switching as it appears in operational SOC chat), evaluated against Qwen2.5-1.5B-Instruct as the "larger general-purpose Spanish model" baseline. This converts Section 9's LATAM-vocabulary claim from qualitative observation to a measured comparison.

12.3 Polish (P2)

Figures (done in this preprint). The `paper/figures/` directory contains four figures: (i) `loss_curve.tex` — loss curve showing the v2 curriculum loss reduction per phase (Figure 2); (ii) `curriculum_schema.pdf` — the curriculum-with-replay schematic (Figure 1); (iii) `b4_density.pdf` — B4 vs. tool-use corpus density sweep (Figure 4); and (iv) `family_scaling.pdf` — B1–B5 across the VectraYX family (Figure 5).

Reproducibility appendix. All training scripts, configuration files, the curriculum sampler, the benchmark harness, the tool-use corpus (`tool_sft_mini_v1.jsonl`), and the B1–B5 evaluation datasets are released at https://github.com/vectrayx/vectrayx-nano-paper. Model checkpoints (Nano 42M post-SFT, Base 260M post-Phase 3, and four Nano LoRA adapters for seeds $\{42, 7, 13, 23\}$) are available at https://huggingface.co/jsantillana/vectrayx-nano. The evaluation datasets are separately released at https://huggingface.co/datasets/jsantillana/vectrayx-bench. A `make repro` target in the repository reproduces the LoRA tool-use experiments end-to-end on a single NVIDIA A10G or L4 GPU.

Safety and red-team study. A small adversarial probe of the model's behavior under offensive-security prompts (unauthorized exploitation, exfiltration, persistence) is required for the public model card. The paper can cite this study as a companion artifact rather than including it in the body.

Energy and carbon reporting. ACM increasingly expects energy reporting per submission. We will translate the existing wall-clock time (∼4 h on L4 for v2 pre-training) into approximate kWh via TDP × utilization × wall-clock and include it in the cost table.

12.4 Beyond this paper

A second-generation VectraYX-Nano v3 would (i) bootstrap on a 50/50 mixture of OpenSubtitles-ES and mC4-ES filtered for short-form prose, (ii) add a DPO [46] stage trained on ∼2,000 chat preferences collected from the human-evaluation panel, and (iii) extend the Phase 3 tooling corpus with multi-step tool-use traces drawn from real MCP-runtime logs. Beyond v3, the natural extension is a continual-pretraining loop driven by the NVD MCP server, in which monthly CVE deltas are folded into a small replay corpus and the model is incrementally re-pretrained without re-running Phases 1–2. We view this as the long-term path to keeping a nano-scale on-prem analyst assistant current with a daily-changing threat landscape.

References

[1] Ehsan Aghaei, Xi Niu, Waseem Shadid, and Ehab Al-Shaer. 2022. SecureBERT: A Domain-Specific Language Model for Cybersecurity. *arXiv preprint arXiv:2204.02685* (2022).

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 4895–4901.

[3] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Cody Blakeney, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Clémentine Lajaunie, Giada Pistilli, Henri Larcher, Leandro von Werra, and Thomas Wolf. 2025. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. *arXiv preprint arXiv:2502.02737* (2025).

[4] Anthropic. 2024. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol. Accessed: 2026-05-08.

[5] AI at Meta. 2024. The Llama 3 Herd of Models. https://ai.meta.com/blog/meta-llama-3/. *Meta AI* (2024).

[6] Markus Bayer, Philipp Kuehn, Ramin Shanehsaz, and Christian Reuter. 2024. CySecBERT: A Domain-Adapted Language Model for the Cybersecurity Domain. *ACM Transactions on Privacy and Security* 27, 2 (2024), 1–20.

[7] Iz Beltagy, Kyle Lo, and Arman Cohan. 2019. SciBERT: A Pretrained Language Model for Scientific Text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. 3615–3620.

[8] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum Learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*. 41–48.

[9] BERTIN Project. 2023. Alpaca-Spanish: Spanish Translation of the Stanford Alpaca Dataset. https://huggingface.co/datasets/bertin-project/alpaca-spanish.

[10] José Cañete, Gabriel Chaperon, Rodrigo Fuentes, Jou-Hui Ho, Hojin Kang, and Jorge Pérez. 2020. Spanish Pre-Trained BERT Model and Evaluation Data. In *PML4DC at ICLR 2020*.

[11] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403* (2023).

[12] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised Cross-lingual Representation Learning at Scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 8440–8451.

[13] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *Advances in Neural Information Processing Systems*, Vol. 36.

[14] Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. 2023. Scaling vision transformers to 22 billion parameters. *arXiv preprint arXiv:2302.05442* (2023).

[15] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. PentestGPT: An LLM-empowered Automatic Penetration Testing Tool. *Proceedings of the 33rd USENIX Security Symposium* (2024).

[16] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv preprint arXiv:2305.14314* (2023).

[17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*.

4171–4186.
[18] Eberhard, David M. and Simons, Gary F. and Fennig, Charles D. (eds.). 2023. Ethnologue: Languages of the World. https://www.ethnologue.com. Online resource.
[19] Robert M French. 1999. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences* 3, 4 (1999), 128–135.
[20] Gemma Team and Gemini Team. 2024. Gemma: Open Models Based on Gemini Research and Technology. *arXiv preprint arXiv:2403.08295* (2024).
[21] Gerganov, Georgi and llama.cpp contributors. 2023. `llama.cpp`: LLM Inference in C/C++. https://github.com/ggerganov/llama.cpp.
[22] Gerganov, Georgi and the ggml contributors. 2024. GGUF: A Unified Binary Format for Quantized Language Models. https://github.com/ggerganov/ggml/blob/master/docs/gguf.md. Accessed: 2026-05-08.
[23] Dirk Groeneveld, Iz Beltagy, Akshita Tsvigun, Ian Magnusson, Yada Wang, Hannaneh Nam, Dustin Schwenk, Mitchell Wortsman, Sameer Bhagia, Oyvind Anas, et al. 2024. OLMo: Accelerating the Science of Language Models. *arXiv preprint arXiv:2402.00838* (2024).
[24] Kshitij Gupta, Benjamin Thérien, Adam Ibrahim, Mats L. Richter, Quentin Anthony, Eugene Belilovsky, Irina Rish, and Timothée Lesort. 2023. Continual Pre-Training of Large Language Models: How to (re)warm your model? *arXiv preprint arXiv:2308.04014* (2023).
[25] Asier Gutiérrez-Fandiño, Jordi Armengol-Estapé, Marc Pàmies, Joan Llop-Palao, Joaquín Silveira-Ocampo, Casimiro Pio Carrino, Carme Armentano-Oller, Carlos Rodríguez-Penagos, Aitor Gonzalez-Agirre, and Marta Villegas. 2022. MarIA: Spanish Language Models. *Procesamiento del Lenguaje Natural* 68 (2022), 39–60.
[26] Asier Gutiérrez-Fandiño, David Pérez-Fernández, Jordi Armengol-Estapé, Aitor Gonzalez-Agirre, and Marta Villegas. 2024. Salamandra: A Spanish & Catalan Language Model Family. *arXiv preprint arXiv:2402.12693* (2024).
[27] Alex Henry, Sainbayar Eavani, Kyunghyun Cho, and Orhan Firat. 2020. Query-Key Normalization for Transformer. *arXiv preprint arXiv:2010.04559* (2020).
[28] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556* (2022).
[29] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685* (2022).
[30] Rawad Ibrahim, Lucas Caccia, Eugene Belilovsky, and Laurent Charlin. 2024. Simple replay buffer is all you need for sparse-reward continual learning. *arXiv preprint arXiv:2402.15795* (2024).
[31] Andrej Karpathy. 2023. nanoGPT. https://github.com/karpathy/nanoGPT.
[32] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences* 114, 13 (2017), 3521–3526.
[33] Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi-Rui Tam, Keith Stevens, Abdullah Barhoum, Nguyen Minh Duc, Oliver Stanley, Richárd Nagy, et al. 2023. Openassistant conversations–democratizing large language model alignment. *arXiv preprint arXiv:2304.07327* (2023).
[34] Taku Kudo and John Richardson. 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*. 66–71.
[35] Pierre Lison and Jörg Tiedemann. 2016. Opensubtitles2016: Extracting large parallel corpora from movie and tv subtitles. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16)*. 923–929.
[36] Xiang Liu, Tianyu Zhou, Guojing Tao, Xiao Liu, Ze Liu, Yuchen Cheng, Sheng Zhang, Yiren Zhang, Muse Chen, Zhaozhuo Chen, et al. 2024. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. *arXiv preprint arXiv:2308.03840* (2024).
[37] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019).
[38] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.
[39] National Institute of Standards and Technology. 2024. National Vulnerability Database (NVD). https://nvd.nist.gov. Accessed: 2026-05-08.
[40] Ollama Team. 2023. Ollama. https://ollama.com.
[41] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[42] Guilherme Penedo, Quentin Malpure, Mohammed Al-Ghosien, Zaid Al-Halah, Adam de Wynter, Shlok Appalaraju, Ragy AlTawy, Sampo Pyysalo, Julien Launay, Yacine Jernite, et al. 2024. The FineWeb dataset. *arXiv preprint arXiv:2406.02029* (2024).
[43] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*.
[44] Qwen Team. 2024. Qwen2. 5: A Family of Large Language Models. *arXiv preprint arXiv:2405.00856* (2024).
[45] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019).
[46] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.
[47] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.
[48] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*. 1715–1725.
[49] Noam Shazeer. 2020. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202* (2020).
[50] Petru Soviany, Radu Tudor Ionescu, Paolo Rota, and Nicu Sebe. 2022. Curriculum Learning: A Survey. *International Journal of Computer Vision* 130, 6 (2022), 1526–1565.
[51] Blake E Strom, Andy Applebaum, Douglas P Miller, Kathryn C Nickels, Adam G Pennington, and Cody B Thomas. 2018. MITRE ATT&CK: Design and Philosophy. In *Proceedings of the 2018 ACM Workshop on Learning from Authoritative Security Data*. 1–11.
[52] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yun Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.
[53] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
[54] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*, Vol. 30.
[55] Wikimedia Foundation. 2001–2024. Wikipedia, the free encyclopedia. https://www.wikipedia.org.
[56] Sang Michael Xie, Hieu Pham, Xinyun Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V. Le, Tengyu Ma, and Adams Wei Yu. 2023. DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.
[57] Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 483–498.
[58] Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in Neural Information Processing Systems* 32 (2019).
[59] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: A small-scale, compute-efficient open-source large language model. *arXiv preprint arXiv:2401.02385* (2024).